Deep Learning

Mathematical Building Blocks of Neural Networks

Prof. Anxiao (Andrew) Jiang

Learning Objectives:

1. Learn to build and train a basic neural network.

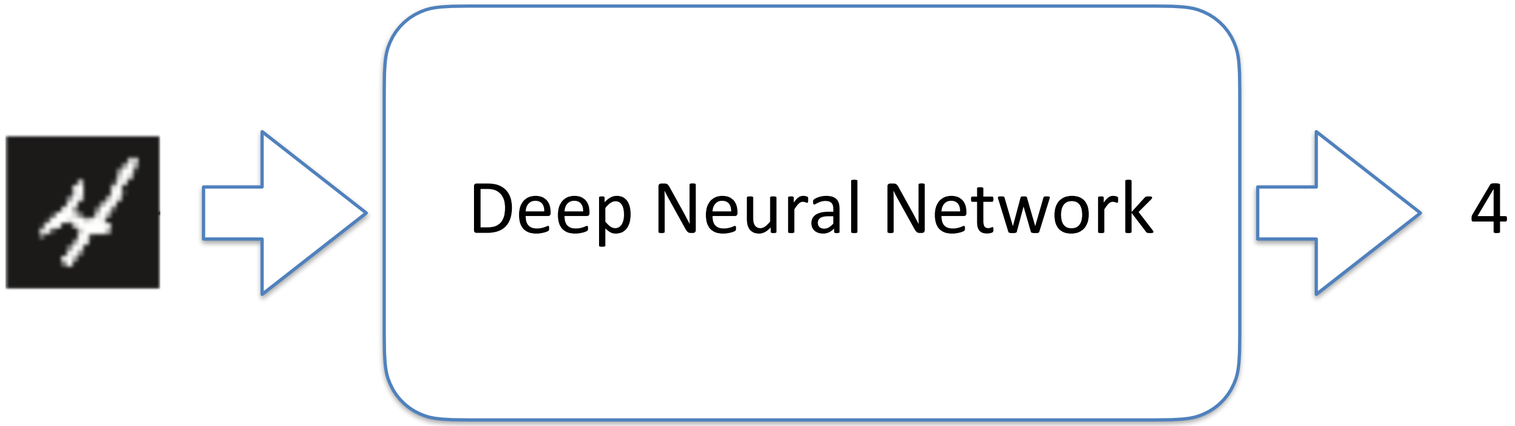
2. Understand basic concepts in deep learning.

Roadmap of this lecture:

1. Handwritten digit recognition.

2. Basic concepts in deep learning.

Handwritten Digit Recognition



28x28 image

10 classes

Step 1: Load the dataset

MNIST Dataset: 60,000 training images and 10,000 test images, along with their labels.

Listing 2.1 Loading the MNIST dataset in Keras

```
from keras.datasets import mnist

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```

Listing 2.1 Loading the MNIST dataset in Keras

```
from keras.datasets import mnist

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```

$60{,}000 \times 28 \times 28$ $10{,}000 \times 28 \times 28$

Each pixel: an integer in [0, 255]

Listing 2.1 Loading the MNIST dataset in Keras

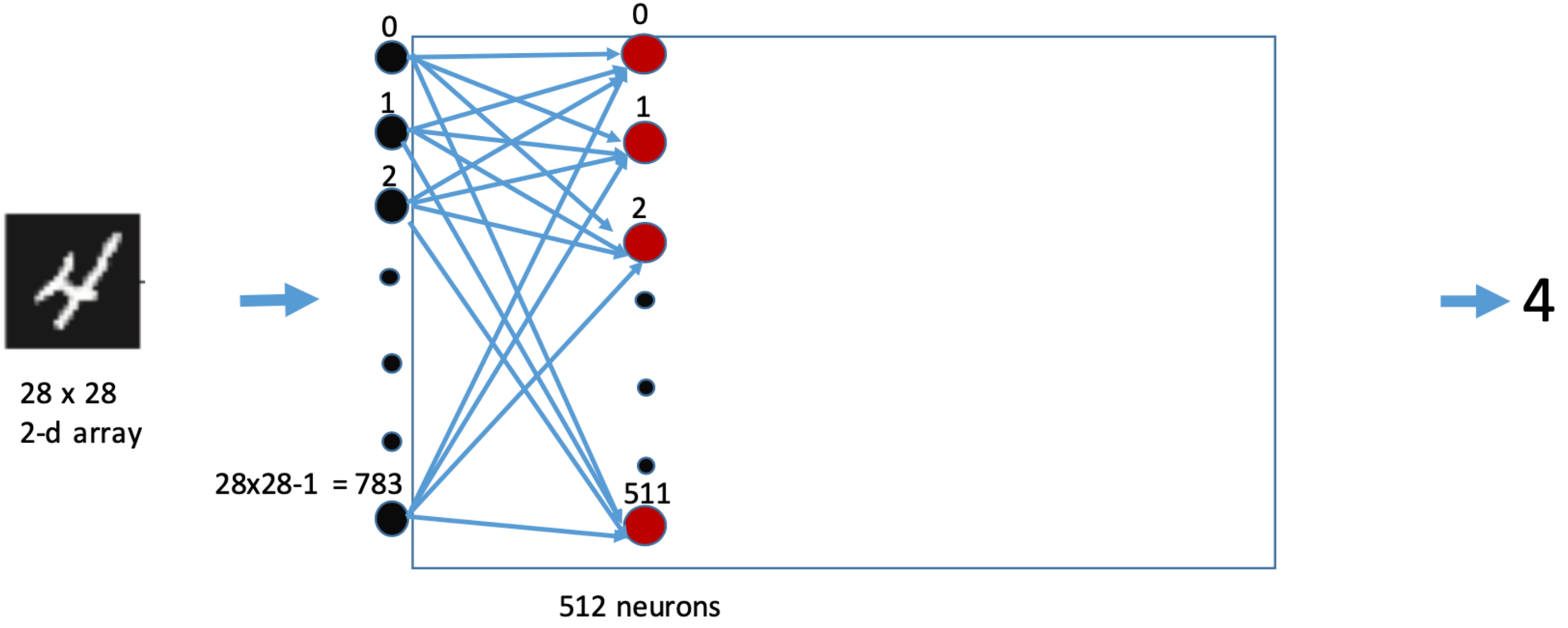
```
from keras.datasets import mnist

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```

60,000 10,000

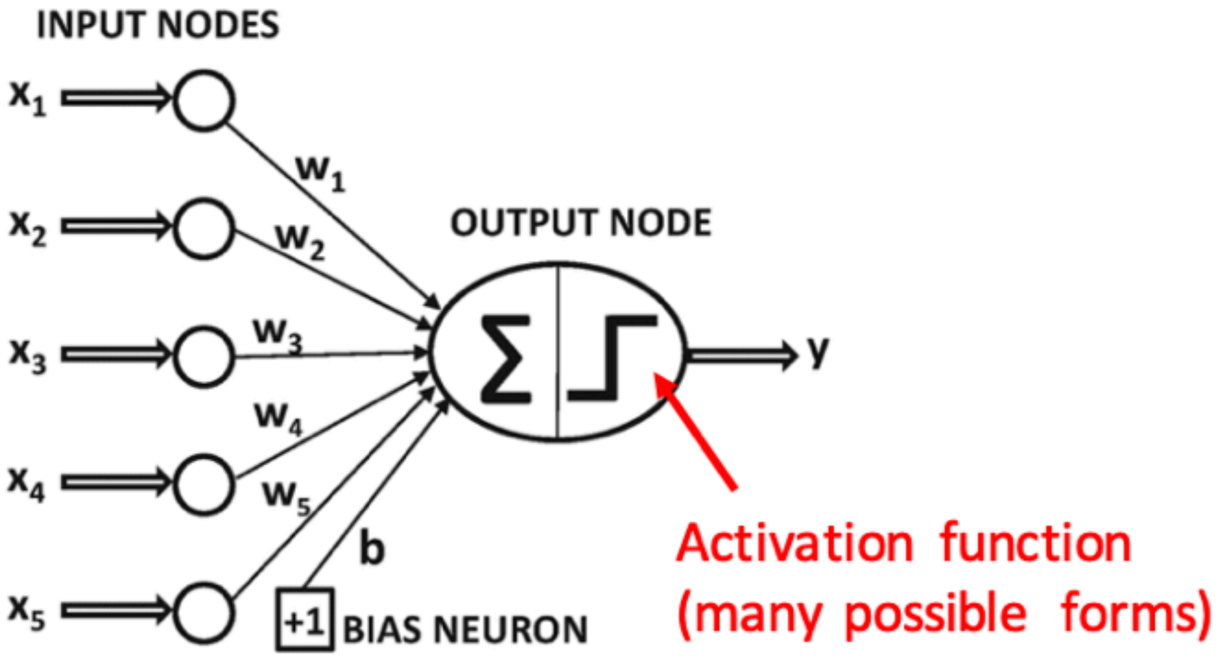
Each label: an integer in [0, 9]

Step 2: Build neural network architecture

```
from keras import models
from keras import layers

network = models.Sequential()
network.add(layers.Dense(512, activation='relu', input_shape=(28 * 28,)))
```



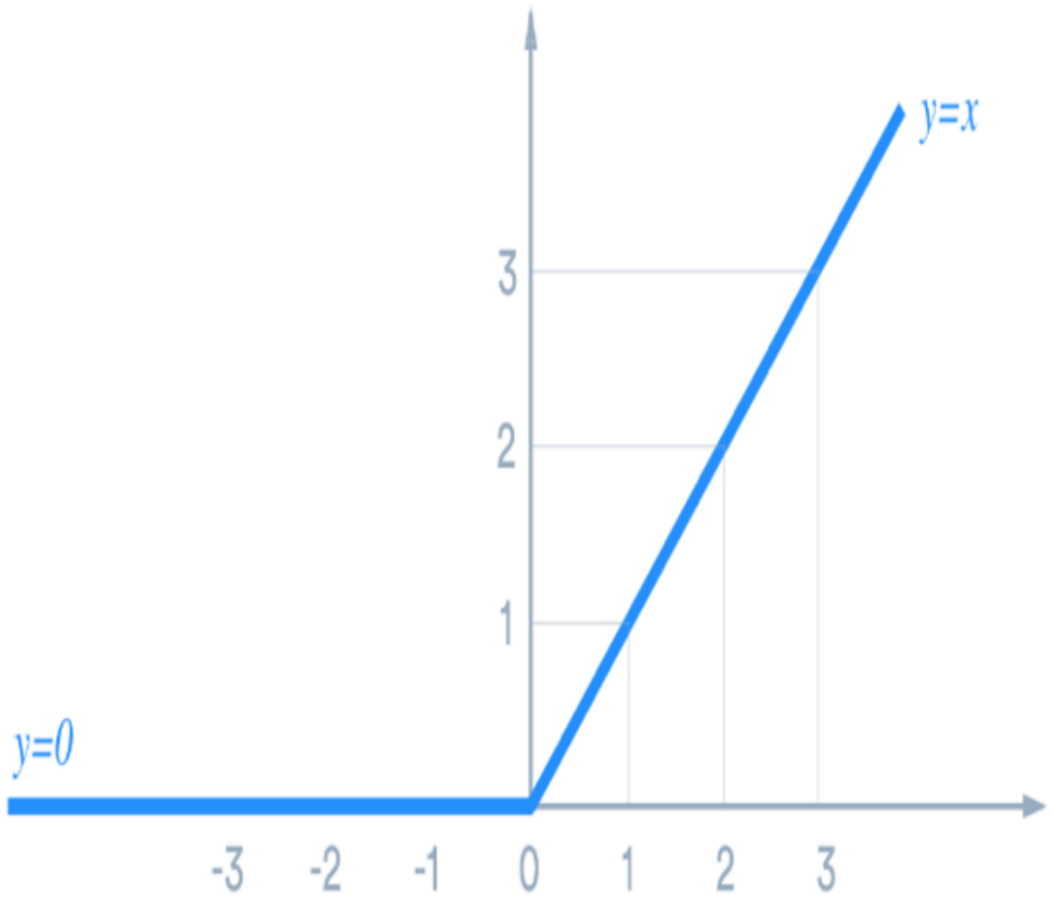
What is a neuron



Perceptron with bias

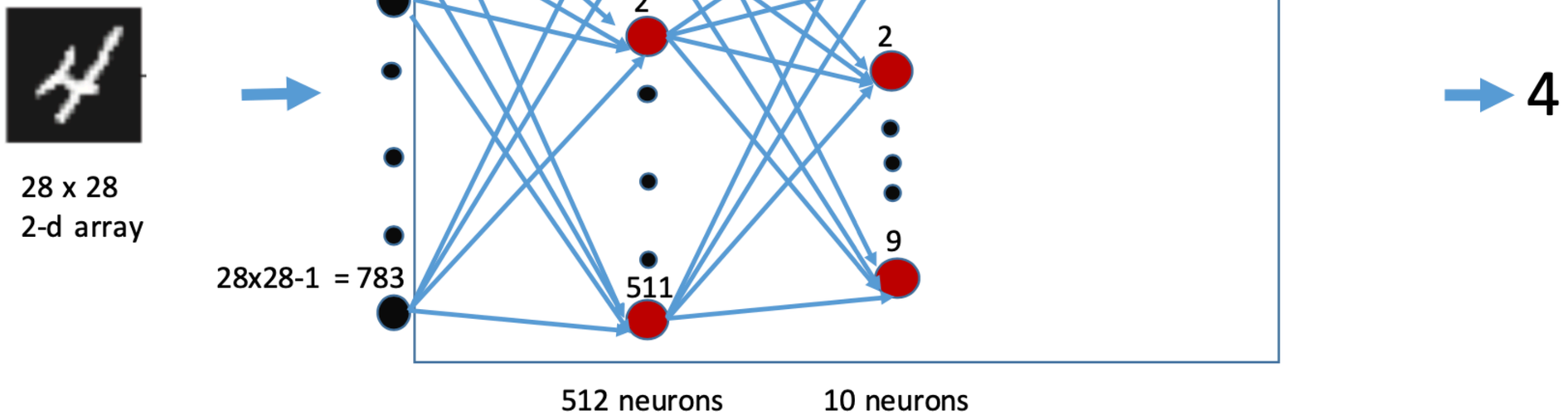
$$y = \text{sign}\{\overline{W} \cdot \overline{X} + b\} = \text{sign}\{\sum_{j=1}^{d} w_j x_j + b\}$$

ReLU

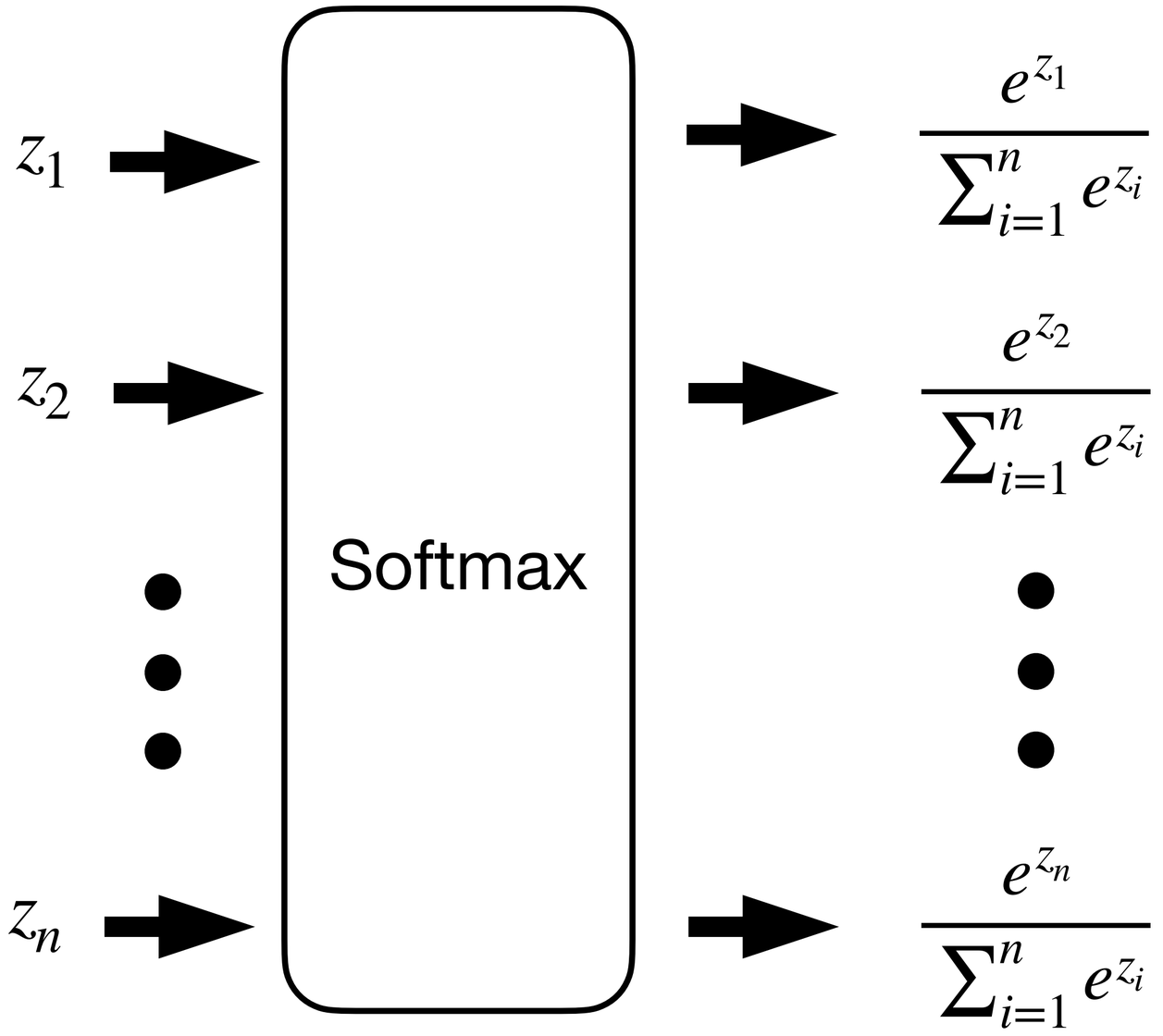


Step 2: Build neural network architecture

```
from keras import models
from keras import layers

network = models.Sequential()
network.add(layers.Dense(512, activation='relu', input_shape=(28 * 28,)))
network.add(layers.Dense(10, activation='softmax'))
```

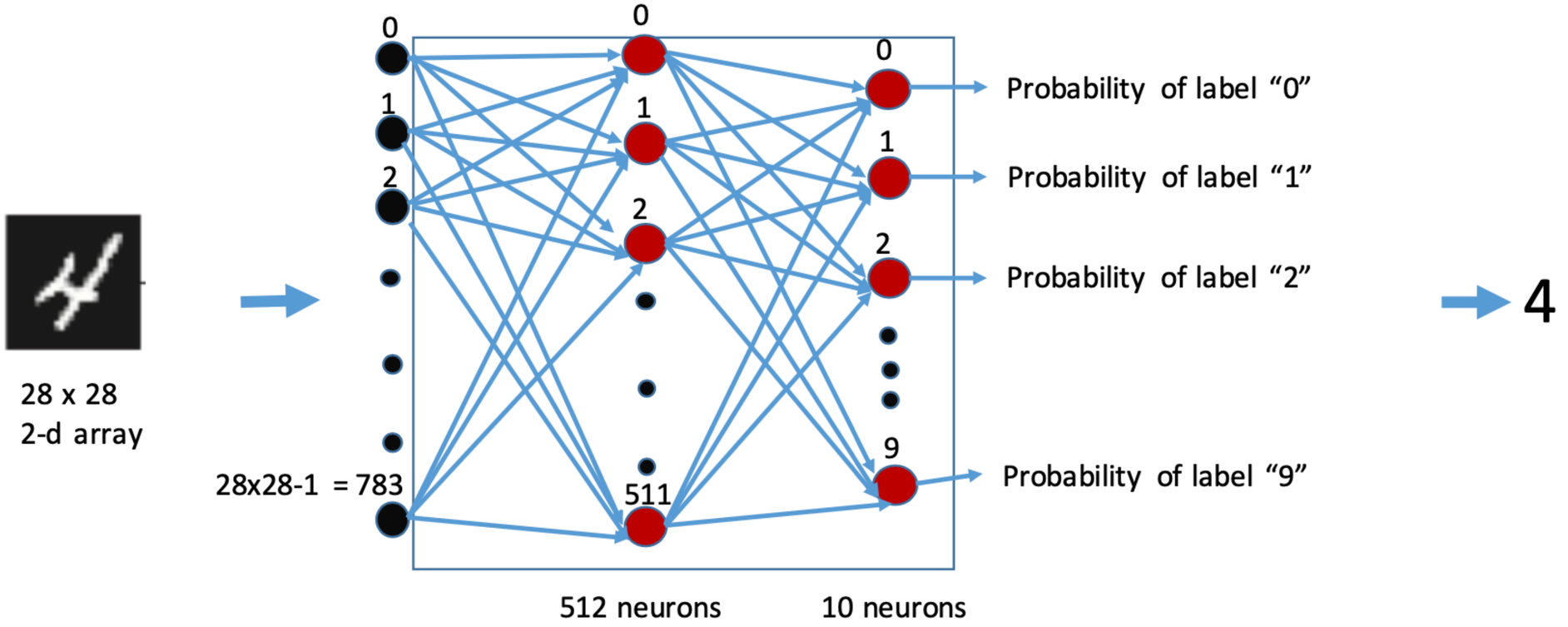


Softmax (an activation function for probabilities)



Step 2: Build neural network architecture

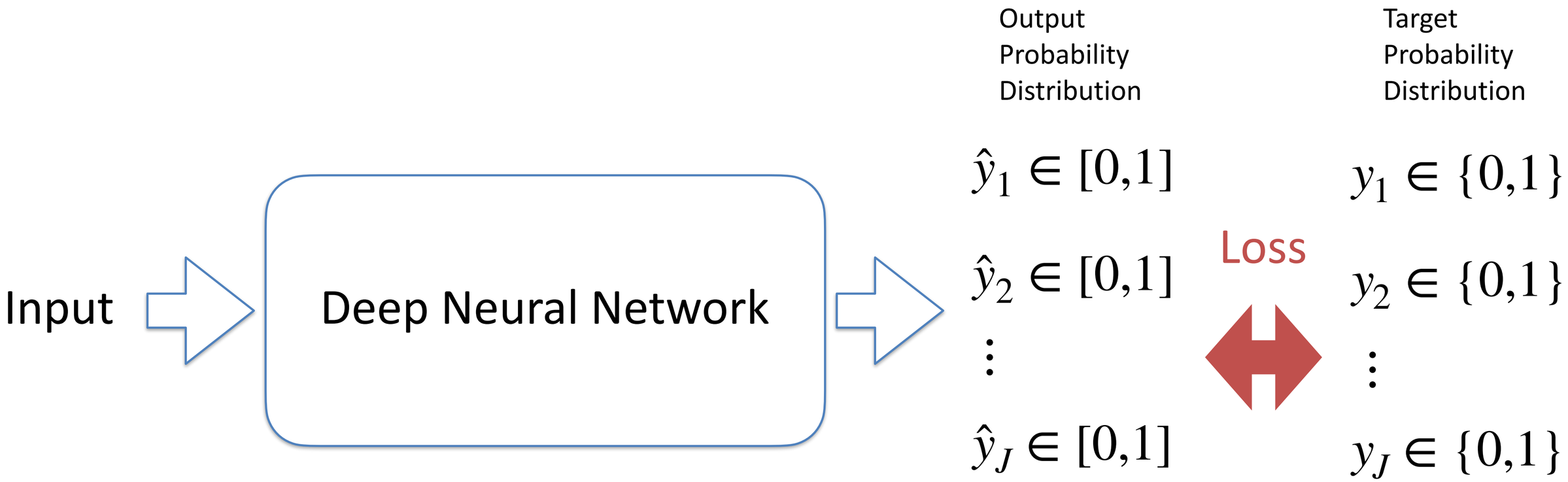
```
from keras import models
from keras import layers

network = models.Sequential()
network.add(layers.Dense(512, activation='relu', input_shape=(28 * 28,)))
network.add(layers.Dense(10, activation='softmax'))
```



Step 3: choose loss function, optimizer, and target metrics

```
network.compile(optimizer='rmsprop',
                loss='categorical_crossentropy',
                metrics=['accuracy'])
```

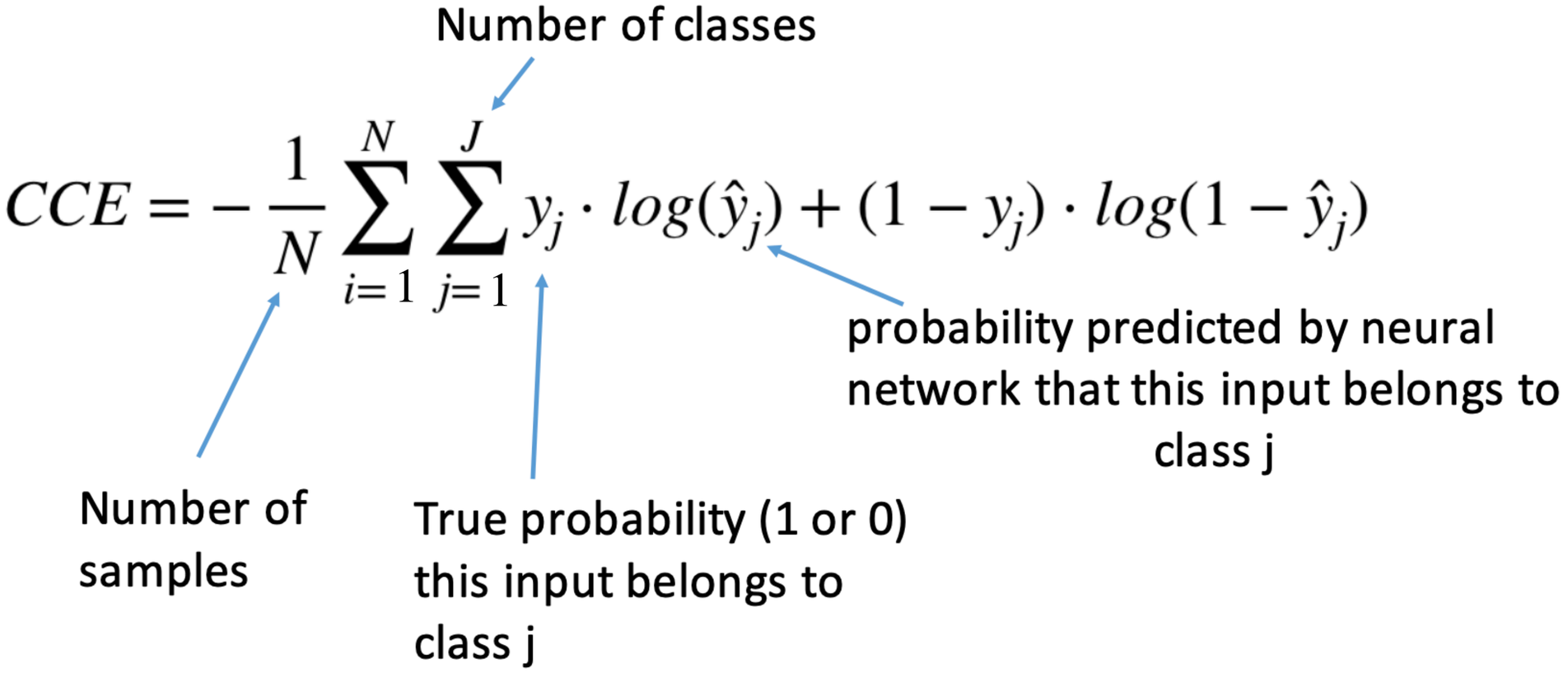
Multi-Class Classification: J Classes



Cross-Entropy: distance between the two distributions $(\hat{y}_1, \hat{y}_2, \cdots, \hat{y}_n)$ and $(y_1, y_2, \cdots, y_n)$.

Cross-Entropy ≥ 0 Cross-Entropy is 0 if and only if the two distributions are the same.

Categorical cross-entropy (a popular loss function for multi-class classification)



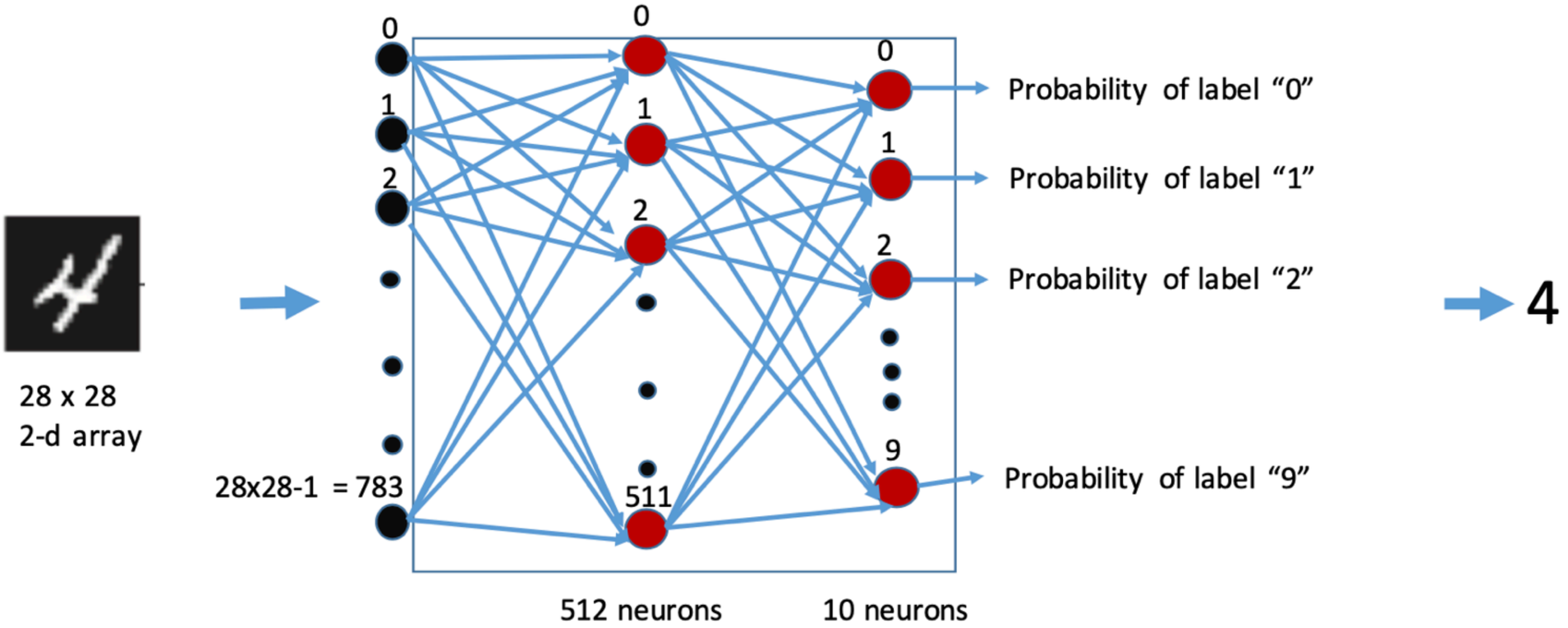
Why is **loss-function** different from **performance metric**?

Categorical cross-entropy

Need to be differentiable

Accuracy

Can be discrete

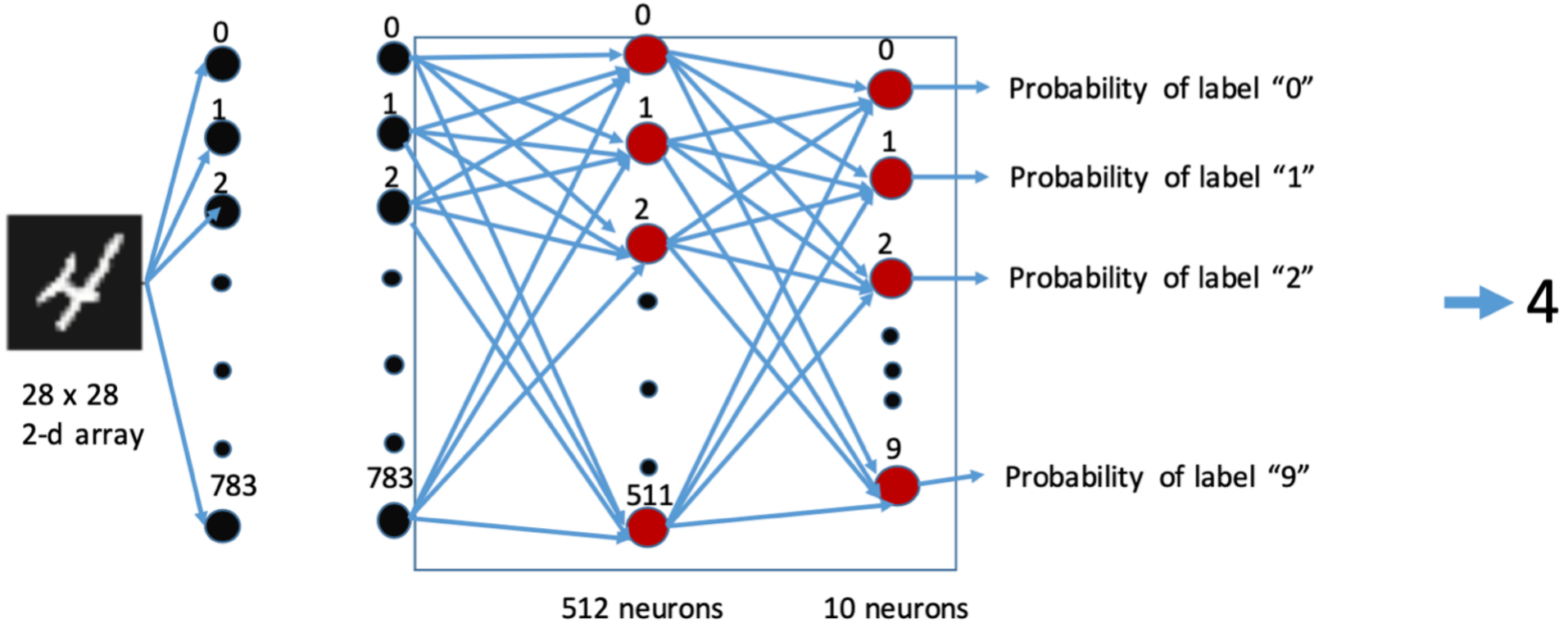

0
1
2
28x28-1 = 783
0
1
2
511
512 neurons
0
1
2
9
10 neurons
Probability of label “0”
Probability of label “1”
Probability of label “2”
Probability of label “9”
28 x 28
2-d array
4

Step 4: Prepare training and test data

Here: Reshape and normalize input training data

Listing 2.4 Preparing the image data

```
train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype('float32') / 255

test_images = test_images.reshape((10000, 28 * 28))
test_images = test_images.astype('float32') / 255
```

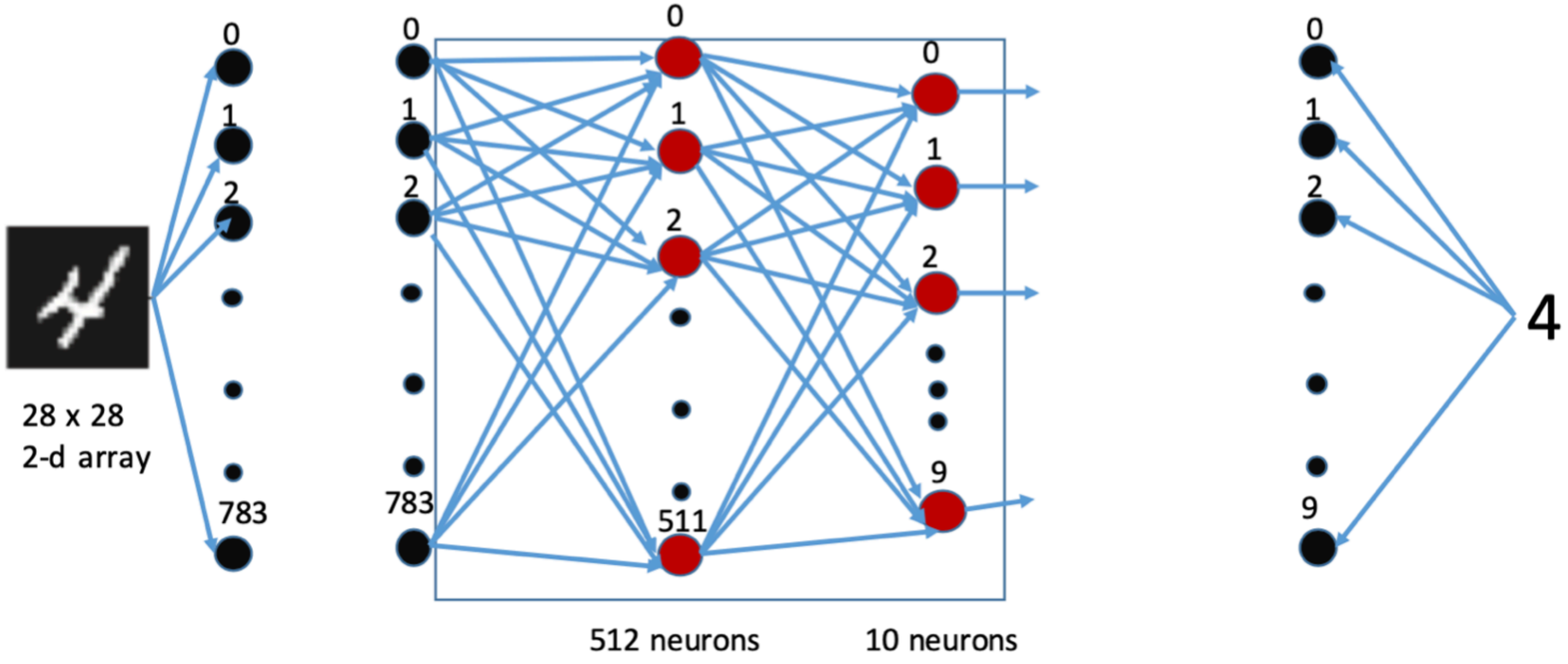

0
1
2
783
28 x 28
2-d array
0
1
2
783
0
1
2
511
0
1
2
9
Probability of label “0”
Probability of label “1”
Probability of label “2”
Probability of label “9”
512 neurons
10 neurons
4

“Reshape” output training data: categorically encode each label using one-hot encoding

```
from keras.utils import to_categorical

train_labels = to_categorical(train_labels)
test_labels = to_categorical(test_labels)
```

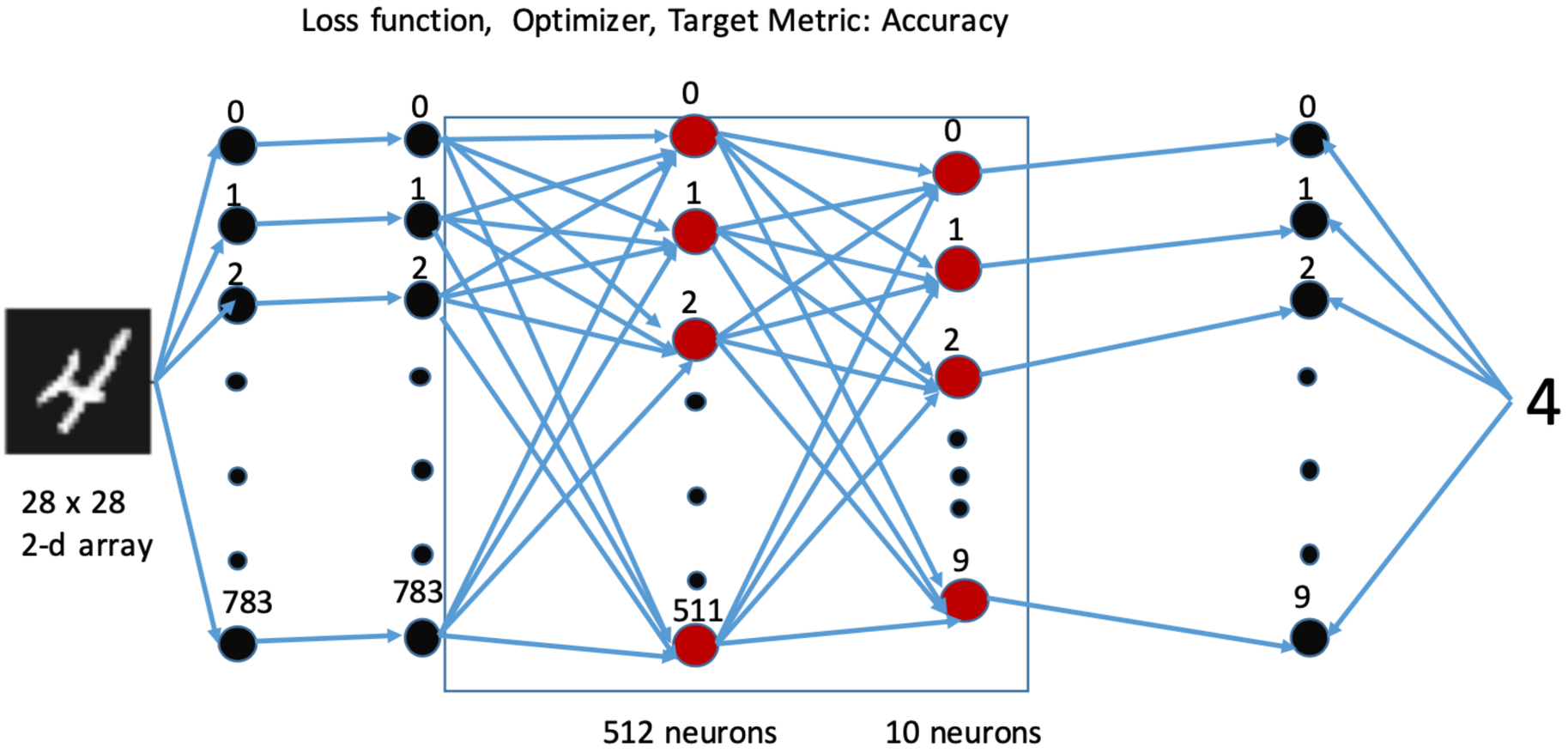
Label	One-hot encoding
0	1,0,0,0,0,0,0,0,0,0
1	0,1,0,0,0,0,0,0,0,0
2	0,0,1,0,0,0,0,0,0,0
3	0,0,0,1,0,0,0,0,0,0
4	0,0,0,0,1,0,0,0,0,0

Label	One-hot encoding
5	0,0,0,0,0,1,0,0,0,0
6	0,0,0,0,0,0,1,0,0,0
7	0,0,0,0,0,0,0,1,0,0
8	0,0,0,0,0,0,0,0,1,0
9	0,0,0,0,0,0,0,0,0,1


0
1
2
783
28 x 28
2-d array
0
1
2
783
0
1
2
511
0
1
2
9
512 neurons
10 neurons
0
1
2
9
4

Step 5: Train the neural network

```
network.fit(train_images, train_labels, epochs=5, batch_size=128)
```



Batch size

The **number of samples** to use in each step of training, for:
(1) computing the loss function and (2) updating the weights.

Epoch

An **Epoch**: all samples in the training set are used once.

Number of epochs: the number of times the training set is used for training.

```
>>> network.fit(train_images, train_labels, epochs=5, batch_size=128)
Epoch 1/5
60000/60000 [==============================] - 9s - loss: 0.2524 - acc: 0.9273
Epoch 2/5
51328/60000 [========================>.....] - ETA: 1s - loss: 0.1035 - acc: 0.9692
```

Step 6: Test the trained neural network

```
>>> test_loss, test_acc = network.evaluate(test_images, test_labels)
>>> print('test_acc:', test_acc)
test_acc: 0.9785
```

Compare to training accuracy: 0.989

Test accuracy is (clearly) lower than training accuracy.
Maybe there is some over-fitting to data.

But still, performance is nice!

Quiz questions:

1. How to build a sequential neural network?
2. How to prepare data for a neural network?
3. What do “epoch” and “batch size” mean?

Roadmap of this lecture:

1. Handwritten digit recognition.

2. Basic concepts in deep learning.

Data representation: Tensor (Array)

- Scalar numbers (0-dimensional tensors)
- Vectors (1-d tensors)
- Matrices (2-d tensors)
- 3-d tensors, and higher-dimensional tensors

Some basic tensor operations

- Add two tesnors (of the same shape): element-wise addition
- Apply a ReLU activation function to a tensor: element-wise operation

Tensor Product

$$X_{2,3,4,5}\ Y_{5,4,7} \ = \ Z_{2,3,4,4,7}$$

$$X = (x_{a,b,c,d})_{2\times3\times4\times5} \qquad Y = (y_{d,e,f})_{5\times4\times7} \qquad Z = (z_{a,b,c,e,f})_{2\times3\times4\times4\times7}$$

$$z_{a,b,c,e,f} = \sum_{d=1}^{5} x_{a,b,c,d}\ y_{d,e,f}$$

Reshape tensor

```
>>> x = np.array([[0., 1.],
                  [2., 3.],
                  [4., 5.]])
>>> print(x.shape)
(3, 2)
```

```
>>> x = x.reshape((6, 1))
>>> x
array([[ 0.],
       [ 1.],
       [ 2.],
       [ 3.],
       [ 4.],
       [ 5.]])
```

```
>>> x = x.reshape((2, 3))
>>> x
array([[ 0.,  1.,  2.],
       [ 3.,  4.,  5.]])
```

GradientTape: Keras API for automatic differentiation

Instantiate a scalar Variable with an initial value of 0.

Open a GradientTape scope.

Inside the scope, apply some tensor operations to our variable.

Use the tape to retrieve the gradient of the output y with respect to our variable x.

```
import tensorflow as tf
x = tf.Variable(0.)
with tf.GradientTape() as tape:
    y = 2 * x + 3
grad_of_y_wrt_x = tape.gradient(y, x)
```

$$y = 2x + 3 \quad \Rightarrow \quad \left.\frac{dy}{dx}\right|_{x=0} = 2$$

```
grad_of_y_wrt_x
```

```
<tf.Tensor: shape=(), dtype=float32, numpy=2.0>
```

GradientTape: Keras API for **automatic differentiation**

```
import tensorflow as tf
x = tf.Variable(3.)
with tf.GradientTape() as tape:
  y = 2 * x * x + 3
grad_of_y_wrt_x = tape.gradient(y,x)
grad_of_y_wrt_x
```

```
<tf.Tensor: shape=(), dtype=float32, numpy=12.0>
```

$$y = 2x^2 + 3 \quad \Rightarrow \quad \left.\frac{dy}{dx}\right|_{x=3} = \left. 4x \right|_{x=3} = 12$$

GradientTape: Keras API for **automatic differentiation**

GradientTape tracks only variables by default. To track a constant, use tape.watch( ).

Listing 3.11 Using `GradientTape` with constant tensor inputs

```
input_const = tf.constant(3.)
with tf.GradientTape() as tape:
    tape.watch(input_const)
    result = tf.square(input_const)
gradient = tape.gradient(result, input_const)
```

$x = \text{input\_const} = 3$ $\quad y = \text{result} = x^2$ $\quad \left.\frac{dy}{dx}\right|_{x=3} = \text{gradient} = 2x\Big|_{x=3} = 6$

GradientTape: Keras API for **automatic differentiation**

Use nested GradientTape to compute second-order gradients

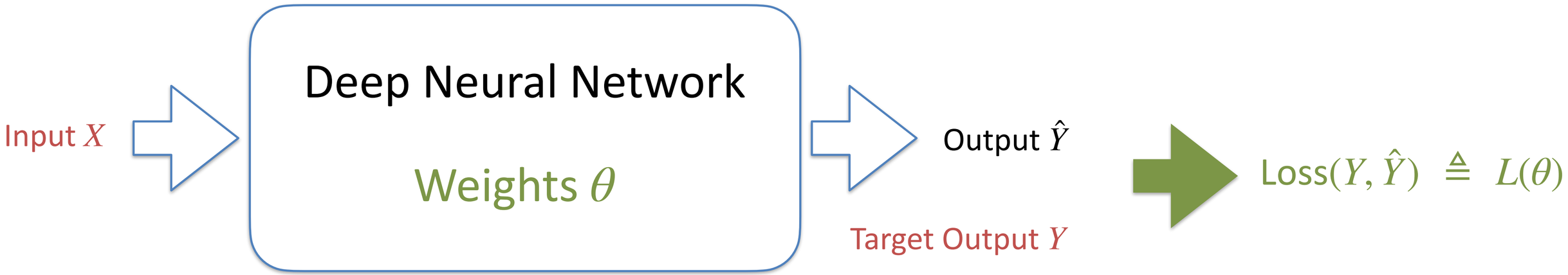
```
time = tf.Variable(0.)
with tf.GradientTape() as outer_tape:
    with tf.GradientTape() as inner_tape:
        position =  4.9 * time ** 2
    speed = inner_tape.gradient(position, time)
acceleration = outer_tape.gradient(speed, time)
```

$x = \text{time} = 0$ $\quad y = \text{position} = 4.9x^2 = 0$ $\quad z = \text{speed} = \left.\frac{dy}{dx}\right|_{x=0} = 9.8x\Big|_{x=0} = 0$

$$\text{acceleration} = \left.\frac{dz}{dx}\right|_{x=0} = \left.\frac{d^2y}{d^2x}\right|_{x=0} = 9.8\Big|_{x=0} = 9.8$$

Backpropagation Algorithm

compute gradient of **loss** with respect to neural network's **weights**



Backpropagation Algorithm

Gradient Descent

Network parameters $\theta = \{w_1, w_2, w_3, \cdots\}$

Loss function $L(\theta) = \text{Loss}(Y, \hat{Y})$

Adjust θ following the gradient $\bigtriangledown L(\theta)$ to minimize $L(\theta)$.

$$\nabla L(\theta) = \begin{pmatrix} \frac{\partial L(\theta)}{\partial w_1} \\ \frac{\partial L(\theta)}{\partial w_2} \\ \frac{\partial L(\theta)}{\partial w_3} \\ \vdots \end{pmatrix}$$

$$\theta = \theta^0$$

$$\theta^1 = \theta^0 - \eta \nabla L(\theta^0)$$

$$\theta^2 = \theta^1 - \eta \nabla L(\theta^1)$$

$$\theta^3 = \theta^2 - \eta \nabla L(\theta^2)$$

$$\vdots$$

The “Backpropagation Algorithm” can compute the gradient efficiently even when there are many (e.g., millions or more) parameters.

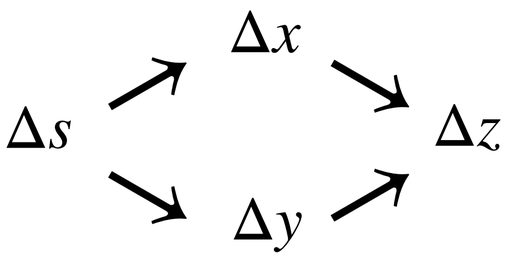
Chain Rules for Gradient

Case 1: $y = f(x)$ $z = g(y)$

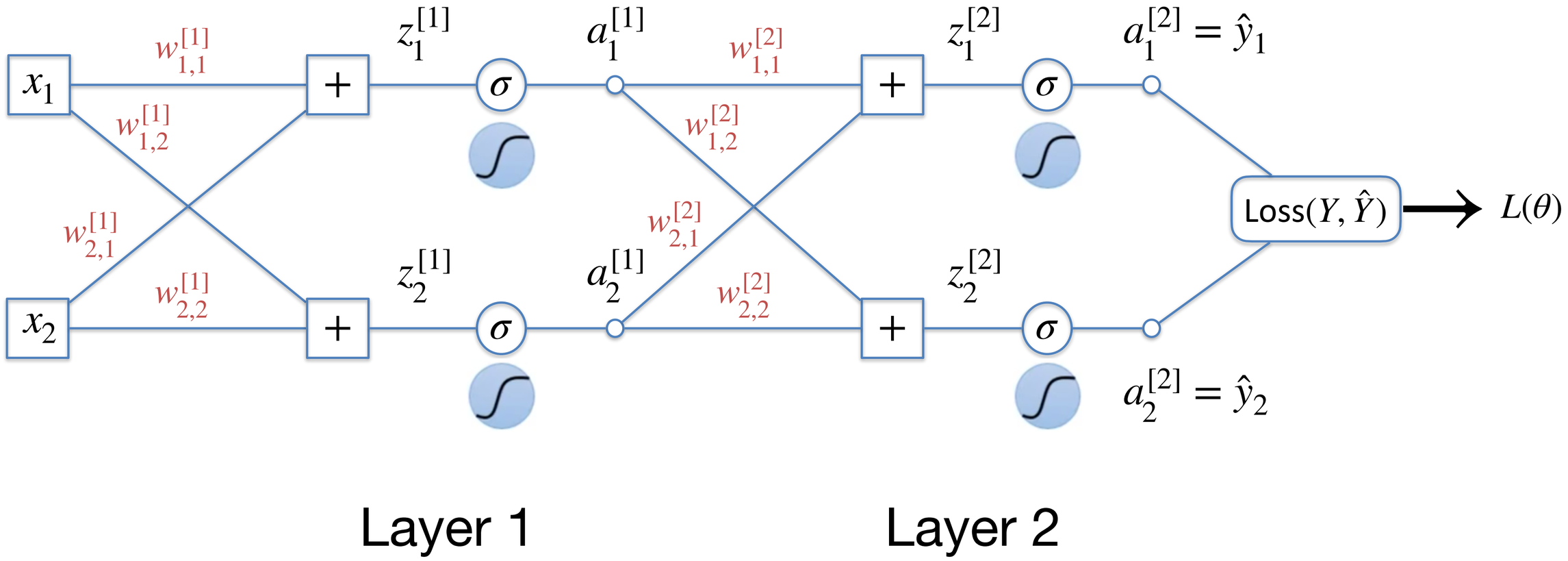
$\Delta x \to \Delta y \to \Delta z$

$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$$

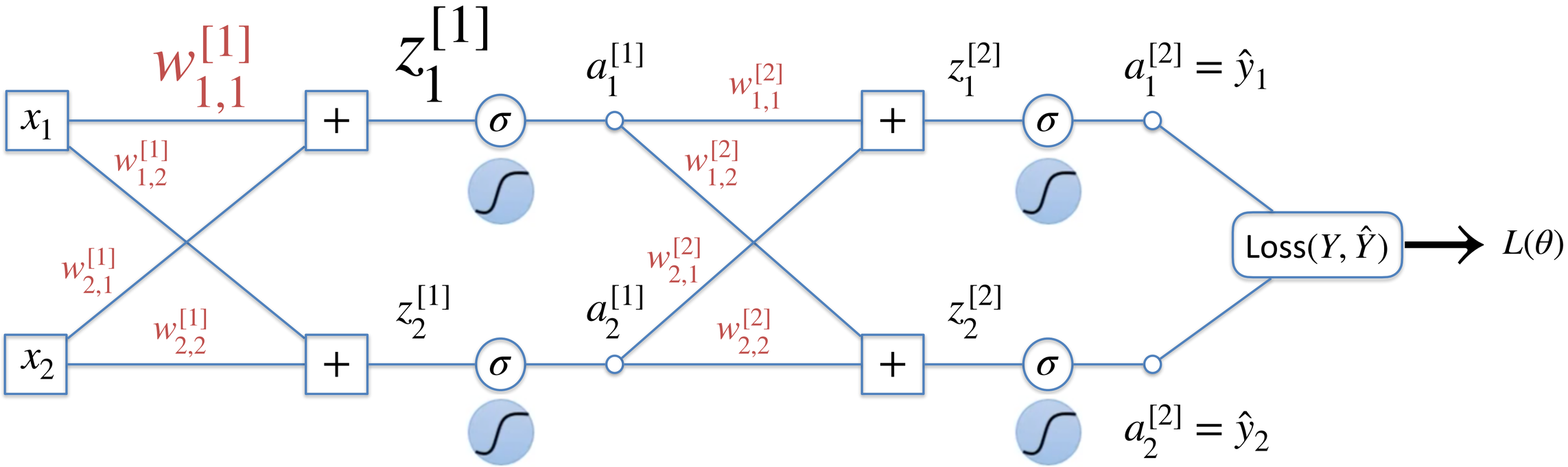
Case 2: $x = f(s)$ $y = g(s)$ $z = h(x, y)$



$$\frac{dz}{ds} = \frac{\partial z}{\partial x}\frac{dx}{ds} + \frac{\partial z}{\partial y}\frac{dy}{ds}$$

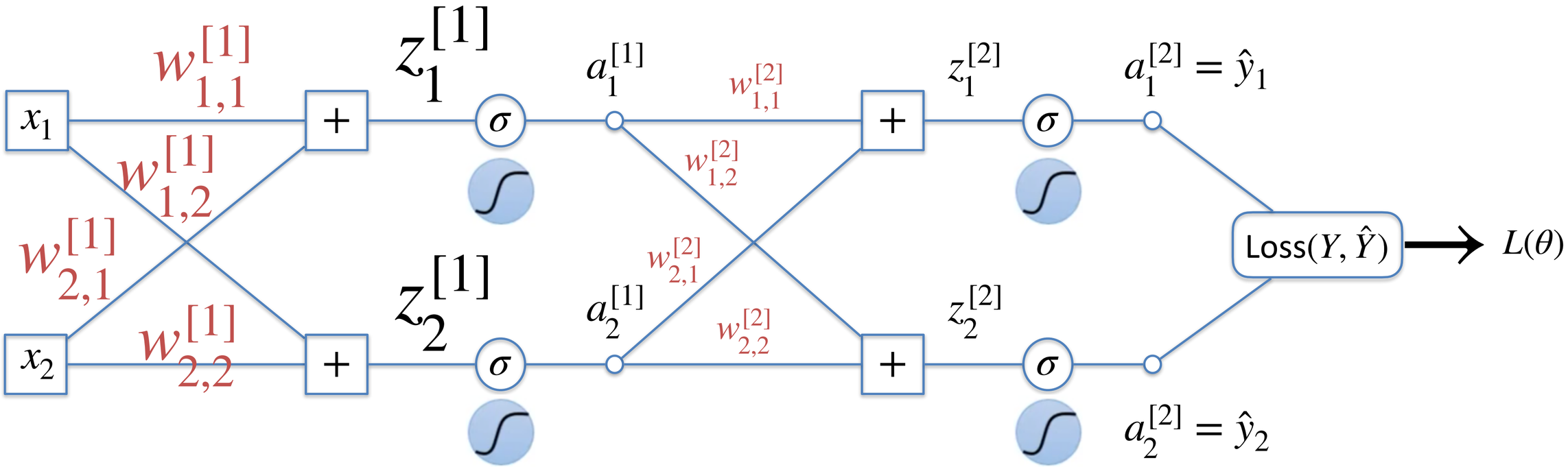

x_1
x_2
$w^{[1]}_{1,1}$
$w^{[1]}_{1,2}$
$w^{[1]}_{2,1}$
$w^{[1]}_{2,2}$
+
+
$z^{[1]}_1$
$z^{[1]}_2$
σ
σ
$a^{[1]}_1$
$a^{[1]}_2$
$w^{[2]}_{1,1}$
$w^{[2]}_{1,2}$
$w^{[2]}_{2,1}$
$w^{[2]}_{2,2}$
+
+
$z^{[2]}_1$
$z^{[2]}_2$
σ
σ
$a^{[2]}_1 = \hat{y}_1$
$a^{[2]}_2 = \hat{y}_2$
Loss$(Y, \hat{Y})$
$L(\theta)$
Layer 1
Layer 2

Forward Pass



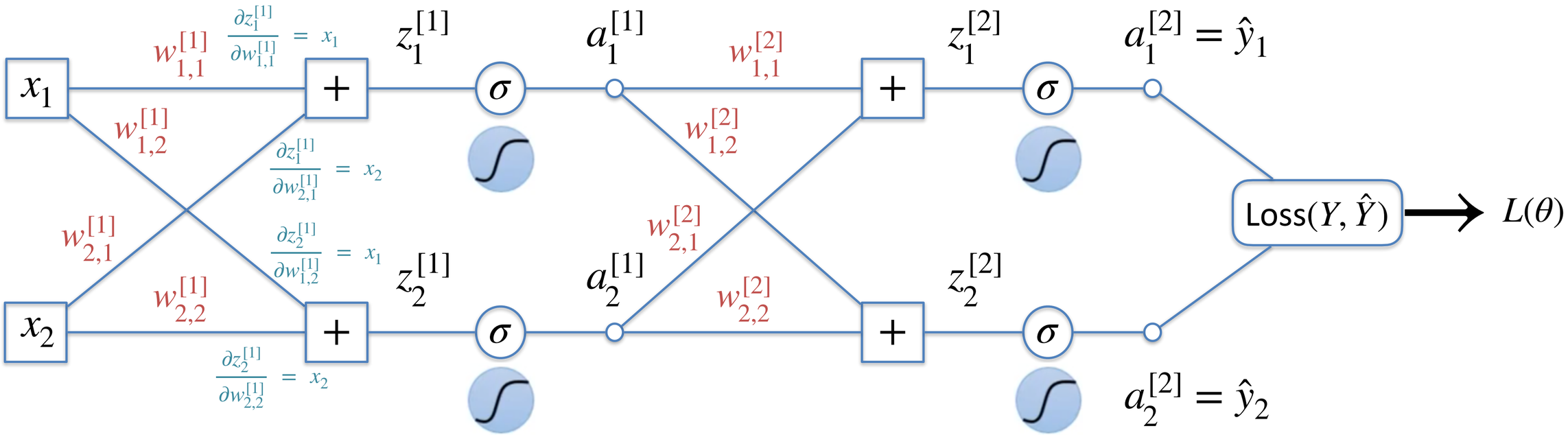
$$z^{[1]}_1 = x_1 w^{[1]}_{1,1} + x_2 w^{[1]}_{2,1} \quad \Rightarrow \quad \frac{\partial z^{[1]}_1}{\partial w^{[1]}_{1,1}} = x_1$$

Forward Pass

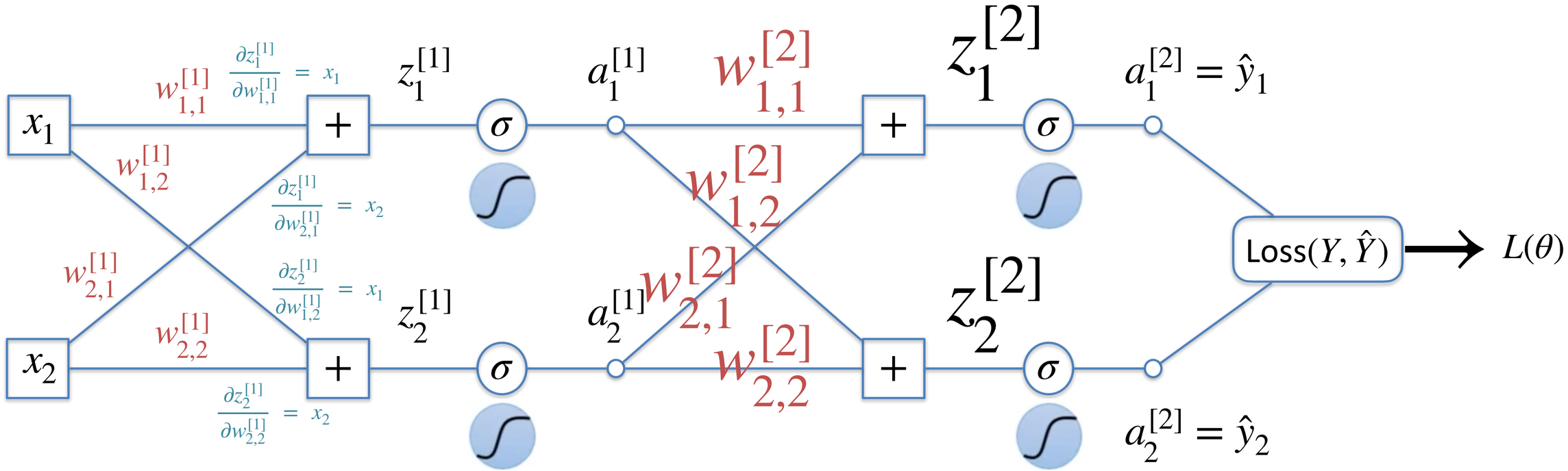


$$\frac{\partial z^{[1]}_1}{\partial w^{[1]}_{1,1}} = x_1 \qquad \frac{\partial z^{[1]}_1}{\partial w^{[1]}_{2,1}} = x_2 \qquad \frac{\partial z^{[1]}_2}{\partial w^{[1]}_{1,2}} = x_1 \qquad \frac{\partial z^{[1]}_2}{\partial w^{[1]}_{2,2}} = x_2$$

Forward Pass

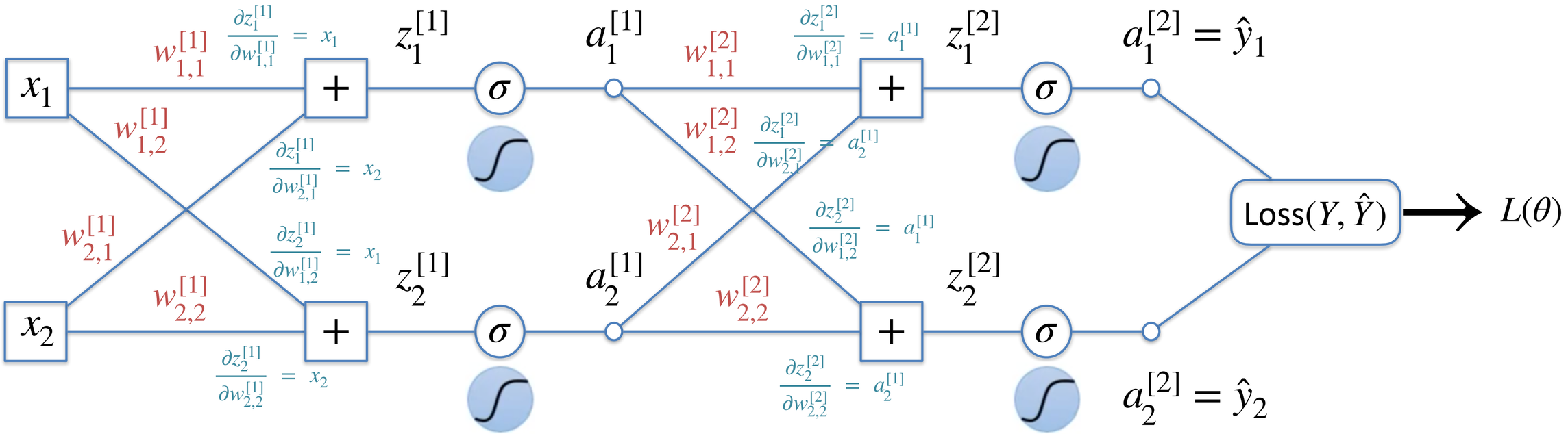


Forward Pass

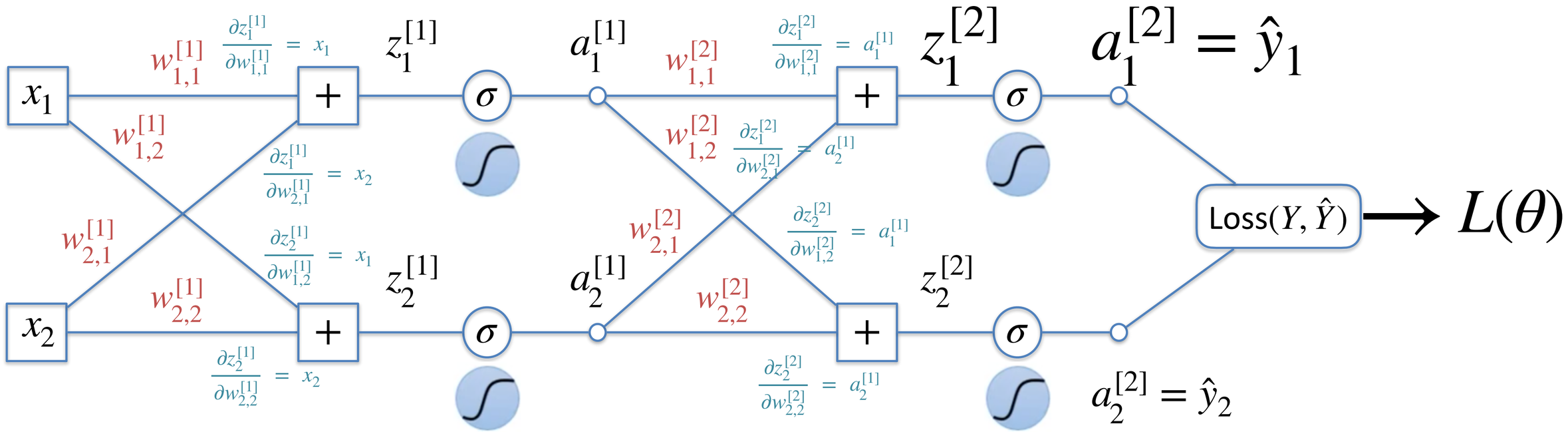


$$\frac{\partial z^{[2]}_1}{\partial w^{[2]}_{1,1}} = a^{[1]}_1 \qquad \frac{\partial z^{[2]}_1}{\partial w^{[2]}_{2,1}} = a^{[1]}_2 \qquad \frac{\partial z^{[2]}_2}{\partial w^{[2]}_{1,2}} = a^{[1]}_1 \qquad \frac{\partial z^{[2]}_2}{\partial w^{[2]}_{2,2}} = a^{[1]}_2$$

Forward Pass

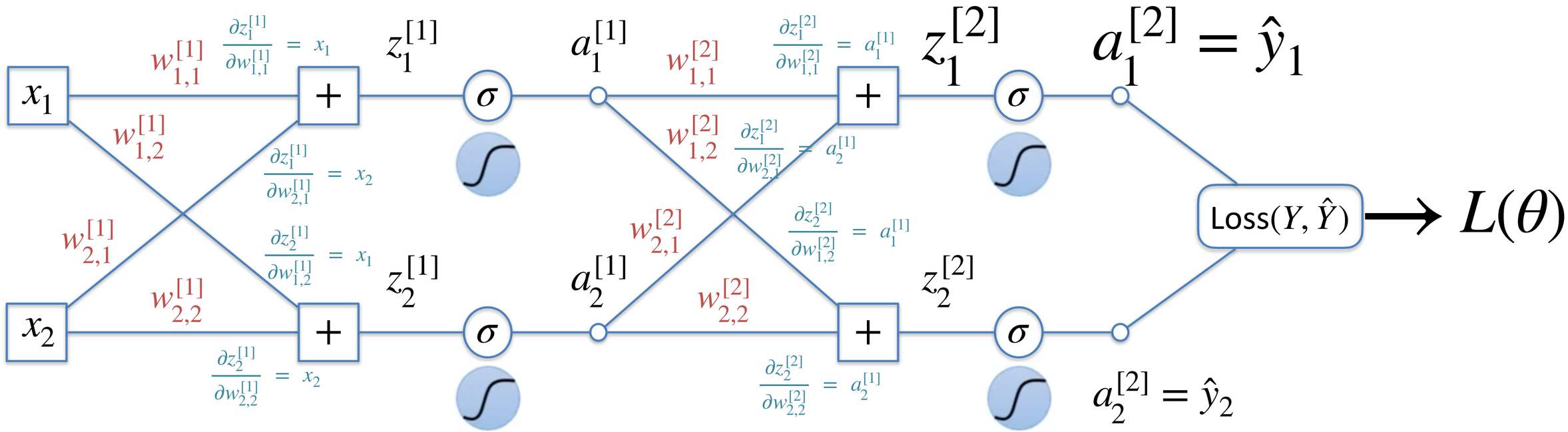


Backward Pass



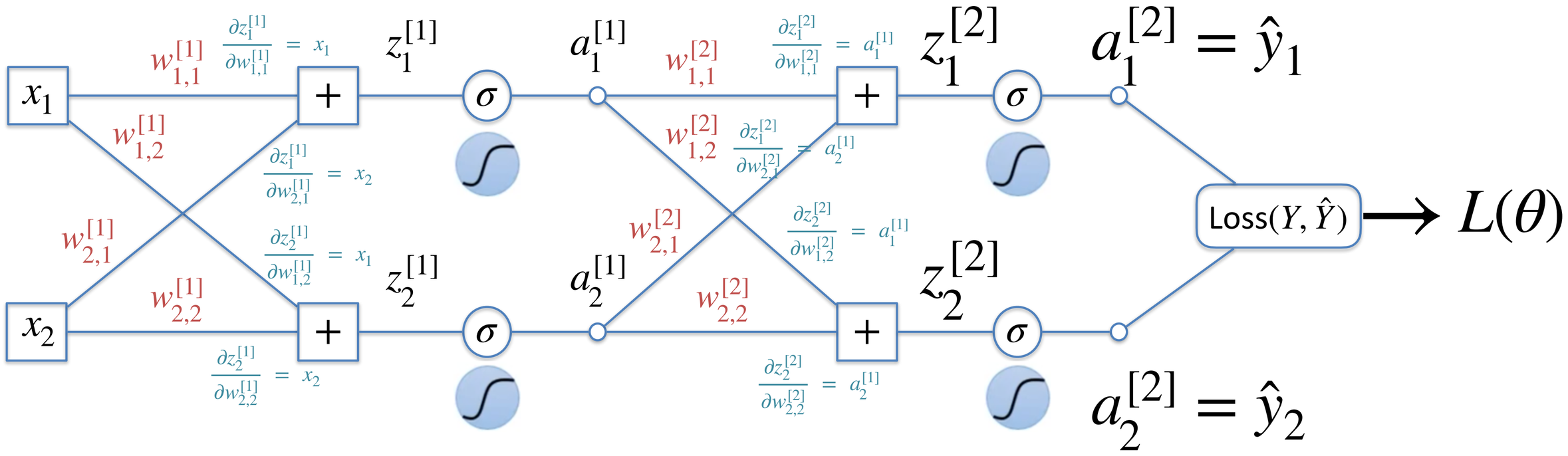
$$\frac{\partial L}{\partial a^{[2]}_1} \qquad a^{[2]}_1 = \sigma\left(z^{[2]}_1\right) \Rightarrow \frac{d\, a^{[2]}_1}{d\, z^{[2]}_1} = \sigma'(z^{[2]}_1)$$

Backward Pass



$$\frac{\partial L}{\partial a_1^{[2]}} \qquad \frac{d\ a_1^{[2]}}{d\ z_1^{[2]}} = \sigma'(z_1^{[2]}) \qquad \Rightarrow \qquad \frac{\partial L}{\partial z_1^{[2]}} = \frac{\partial L}{\partial a_1^{[2]}} \cdot \frac{d\ a_1^{[2]}}{d\ z_1^{[2]}} = \frac{\partial L}{\partial a_1^{[2]}} \cdot \sigma'(z_1^{[2]})$$

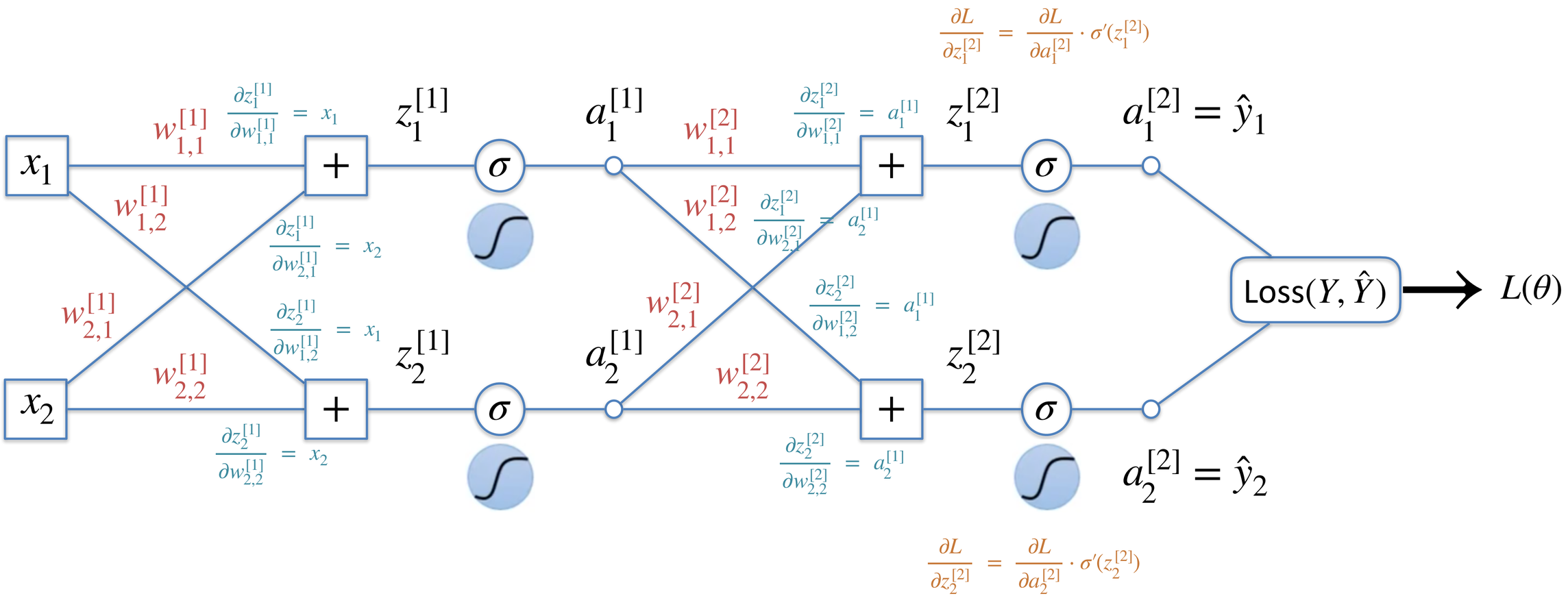
Backward Pass



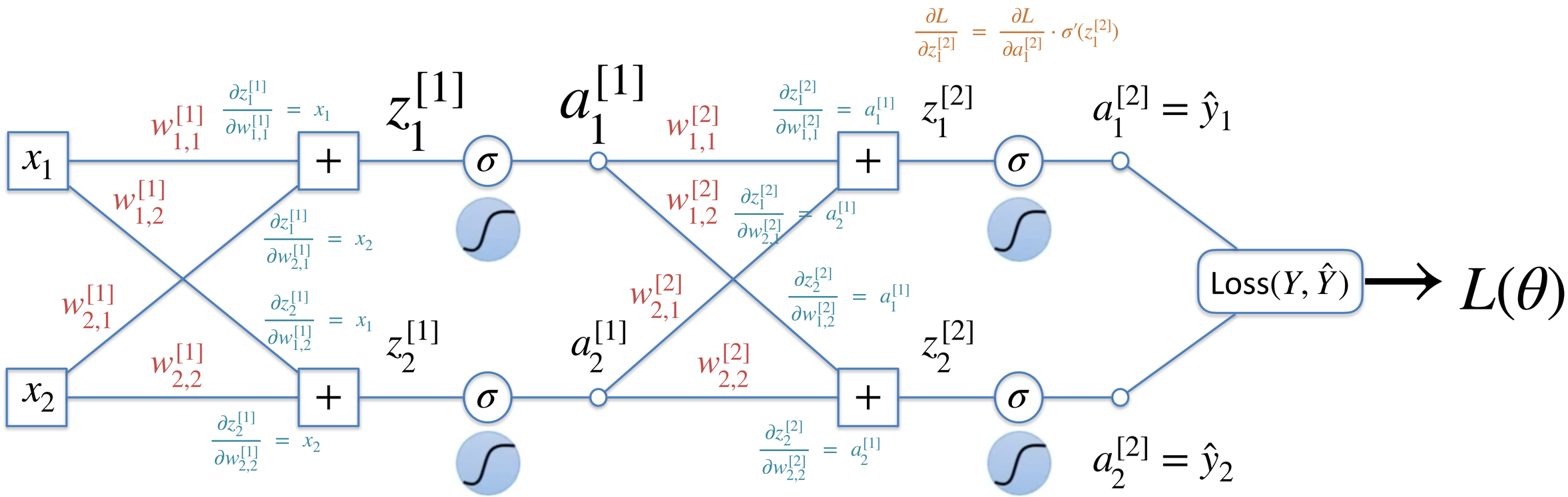
$$\frac{\partial L}{\partial z^{[2]}_1} = \frac{\partial L}{\partial a^{[2]}_1} \cdot \sigma'(z^{[2]}_1)$$

$$\frac{\partial L}{\partial z^{[2]}_2} = \frac{\partial L}{\partial a^{[2]}_2} \cdot \sigma'(z^{[2]}_2)$$

Backward Pass

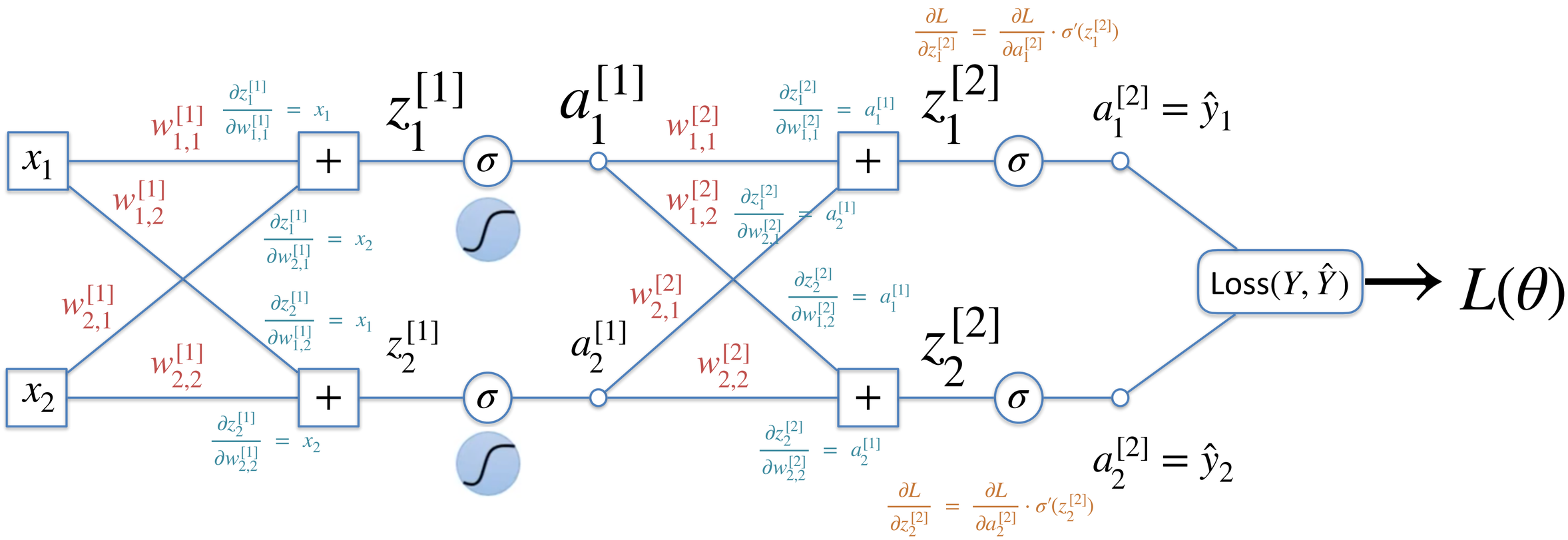


Backward Pass



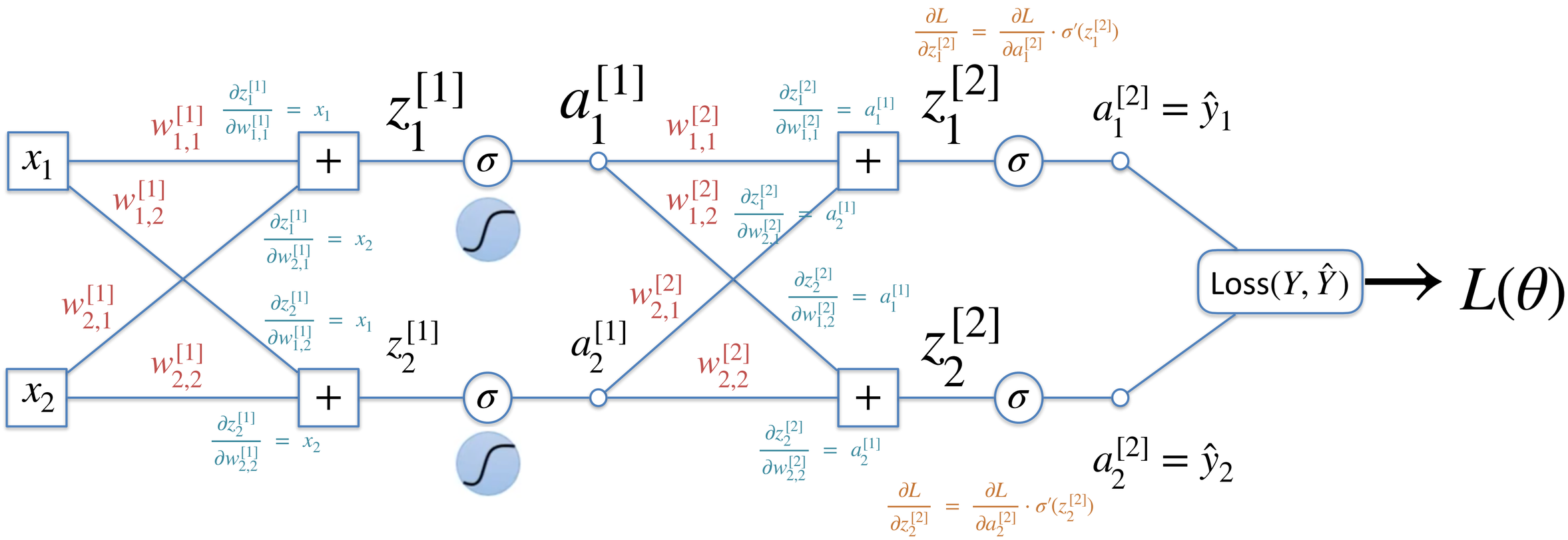
$$\frac{\partial L}{\partial z_1^{[1]}} = \frac{\partial L}{\partial a_1^{[1]}} \cdot \frac{d\ a_1^{[1]}}{d\ z_1^{[1]}} = \frac{\partial L}{\partial a_1^{[1]}} \cdot \sigma'(z_1^{[1]})$$

Backward Pass



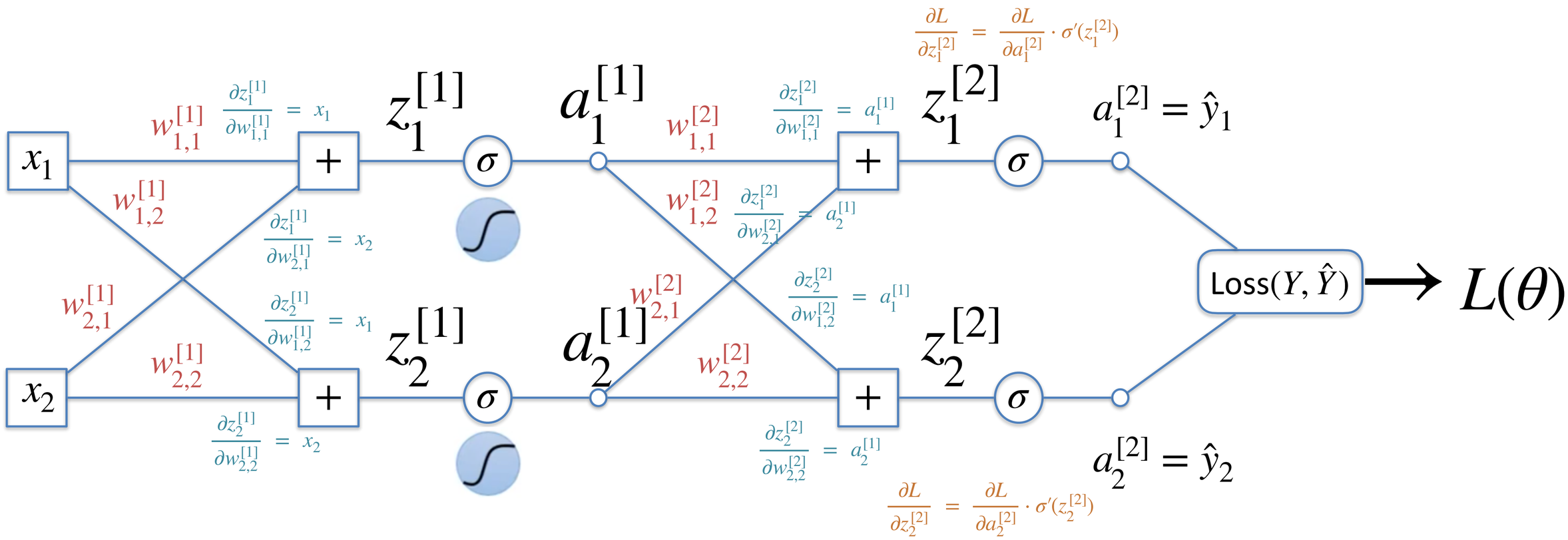
$$\frac{\partial L}{\partial z^{[1]}_1} = \frac{\partial L}{\partial a^{[1]}_1} \cdot \sigma'(z^{[1]}_1) = \left(\frac{\partial L}{\partial z^{[2]}_1} \cdot \frac{\partial z^{[2]}_1}{\partial a^{[1]}_1} + \frac{\partial L}{\partial z^{[2]}_2} \cdot \frac{\partial z^{[2]}_2}{\partial a^{[1]}_1} \right) \cdot \sigma'(z^{[1]}_1)$$

Backward Pass



$$\frac{\partial L}{\partial z^{[1]}_1} = \left(\frac{\partial L}{\partial z^{[2]}_1} \cdot \frac{\partial z^{[2]}_1}{\partial a^{[1]}_1} + \frac{\partial L}{\partial z^{[2]}_2} \cdot \frac{\partial z^{[2]}_2}{\partial a^{[1]}_1} \right) \cdot \sigma'(z^{[1]}_1) = \left(\frac{\partial L}{\partial z^{[2]}_1} \cdot w^{[2]}_{1,1} + \frac{\partial L}{\partial z^{[2]}_2} \cdot w^{[2]}_{1,2} \right) \cdot \sigma'(z^{[1]}_1)$$

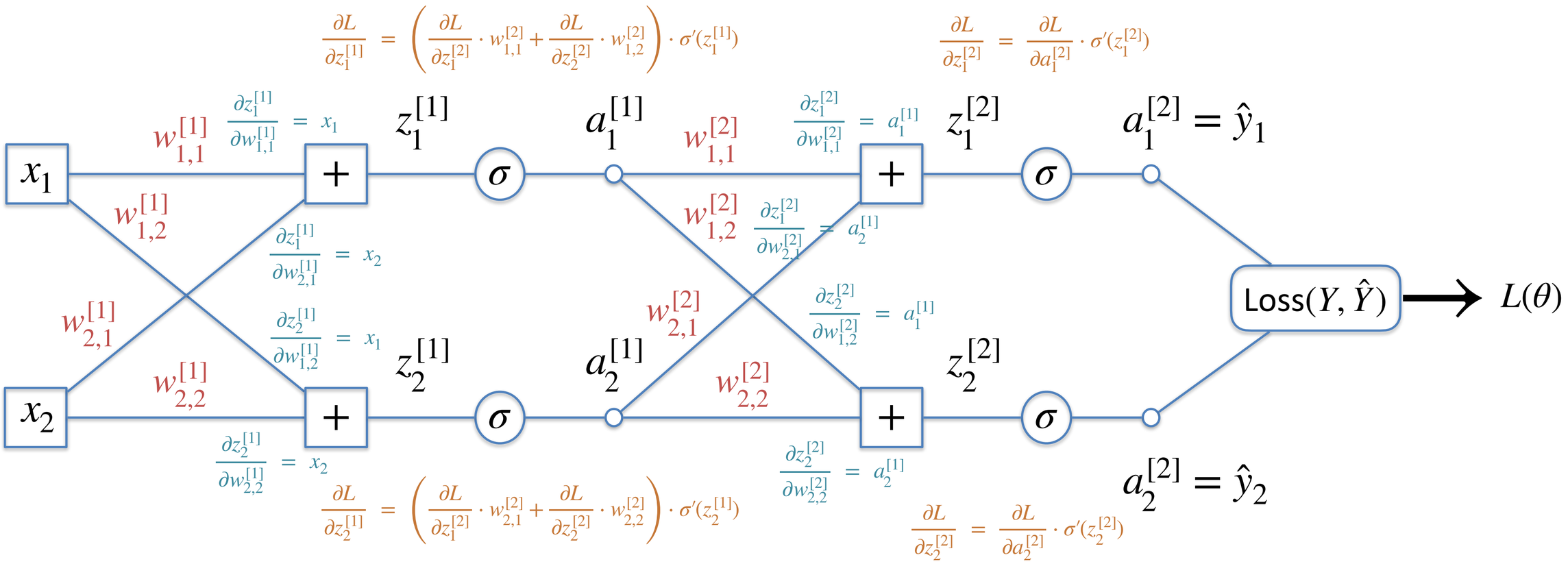
Backward Pass



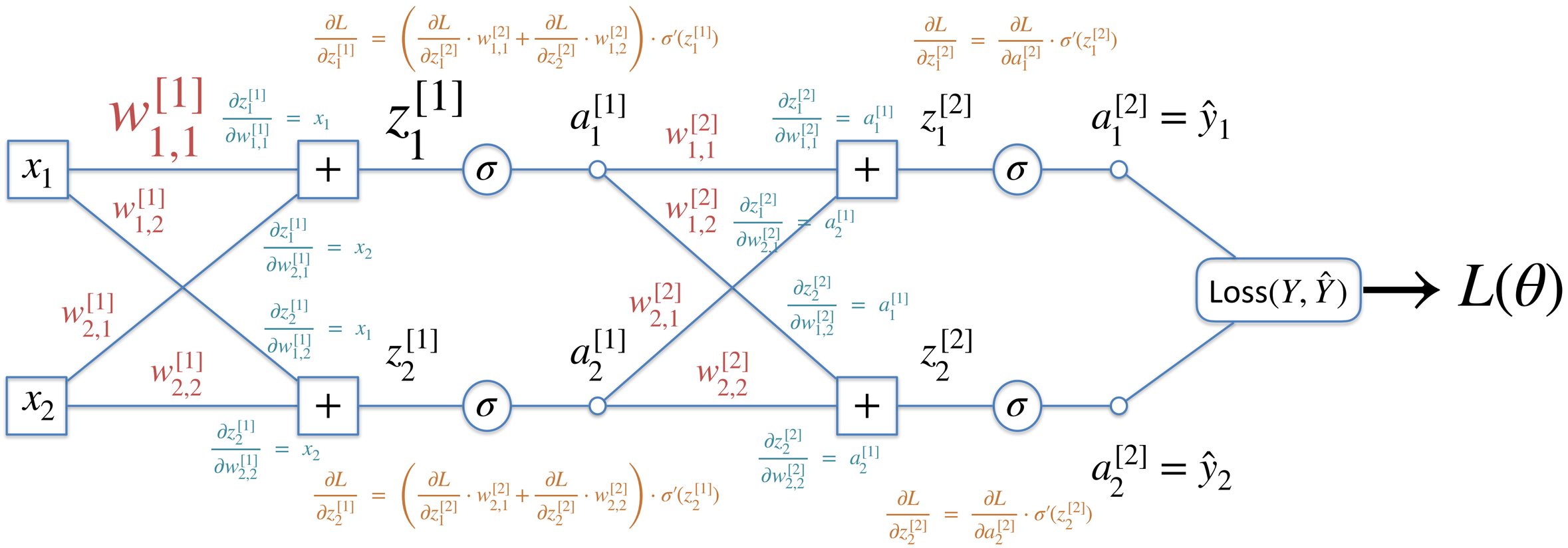
$$\frac{\partial L}{\partial z^{[1]}_1} = \left(\frac{\partial L}{\partial z^{[2]}_1} \cdot w^{[2]}_{1,1} + \frac{\partial L}{\partial z^{[2]}_2} \cdot w^{[2]}_{1,2} \right) \cdot \sigma'(z^{[1]}_1)$$

$$\frac{\partial L}{\partial z^{[1]}_2} = \left(\frac{\partial L}{\partial z^{[2]}_1} \cdot w^{[2]}_{2,1} + \frac{\partial L}{\partial z^{[2]}_2} \cdot w^{[2]}_{2,2} \right) \cdot \sigma'(z^{[1]}_2)$$

Backward Pass

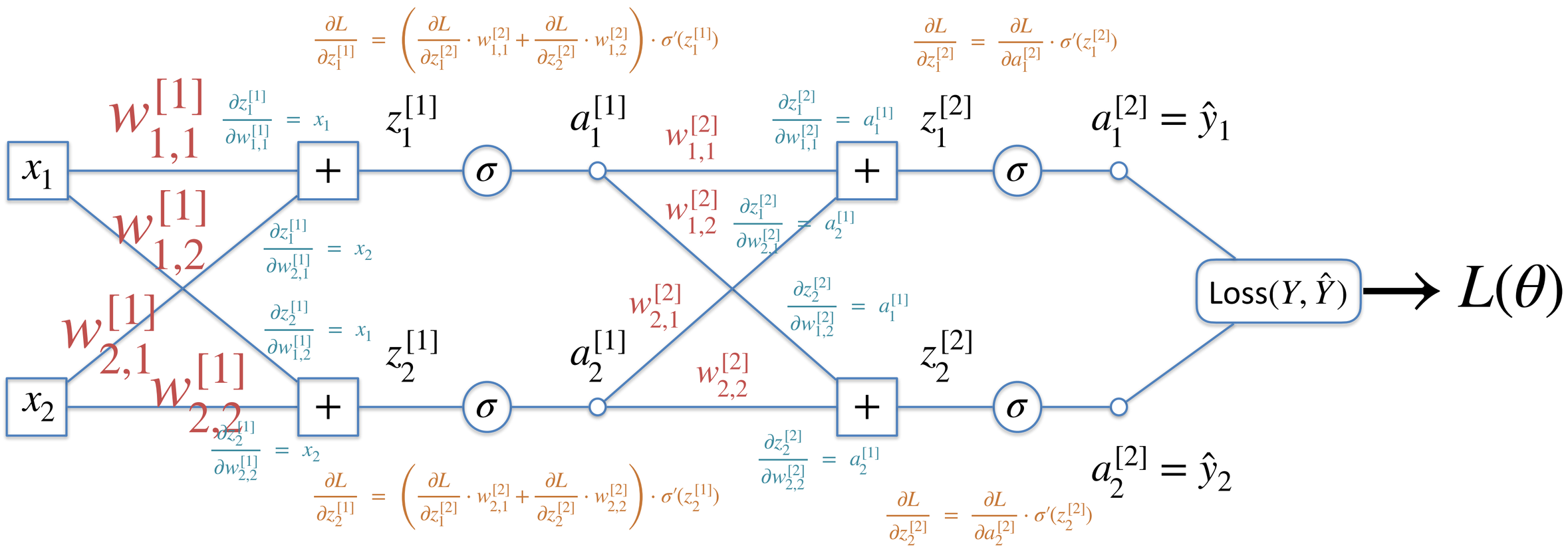


Compute Gradient



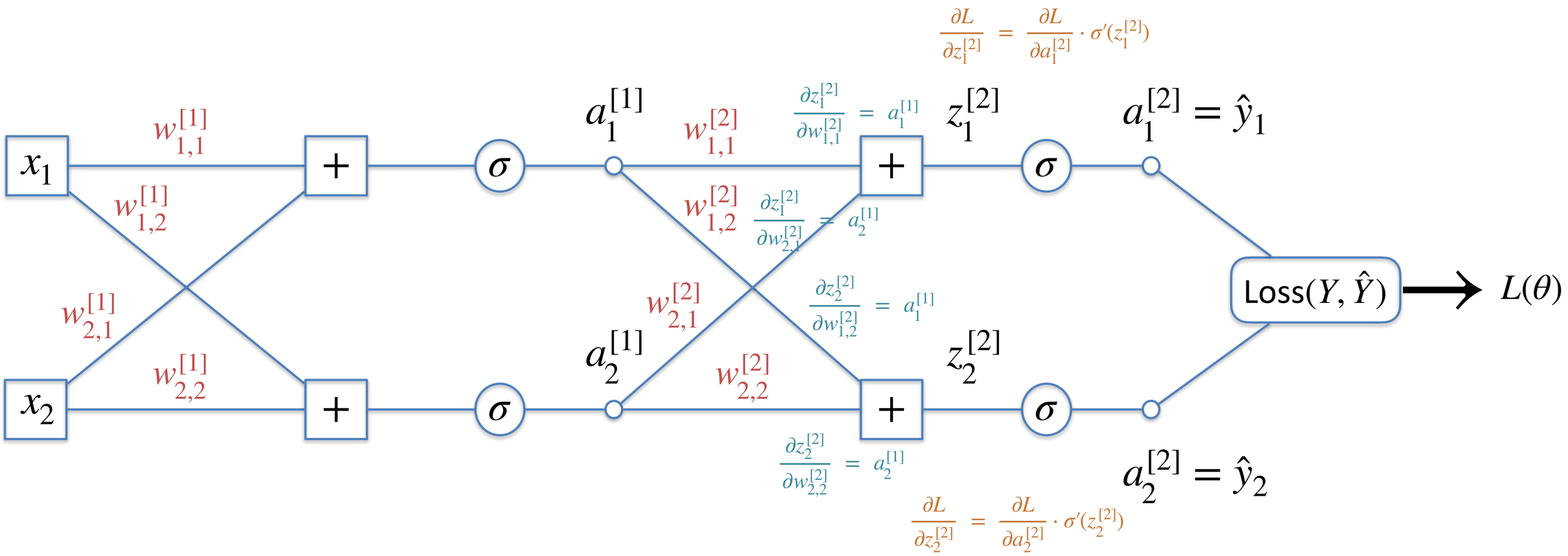
$$\frac{\partial L}{\partial w_{1,1}^{[1]}} = \frac{\partial L}{\partial z_1^{[1]}} \cdot \frac{\partial z_1^{[1]}}{\partial w_{1,1}^{[1]}}$$

Compute Gradient



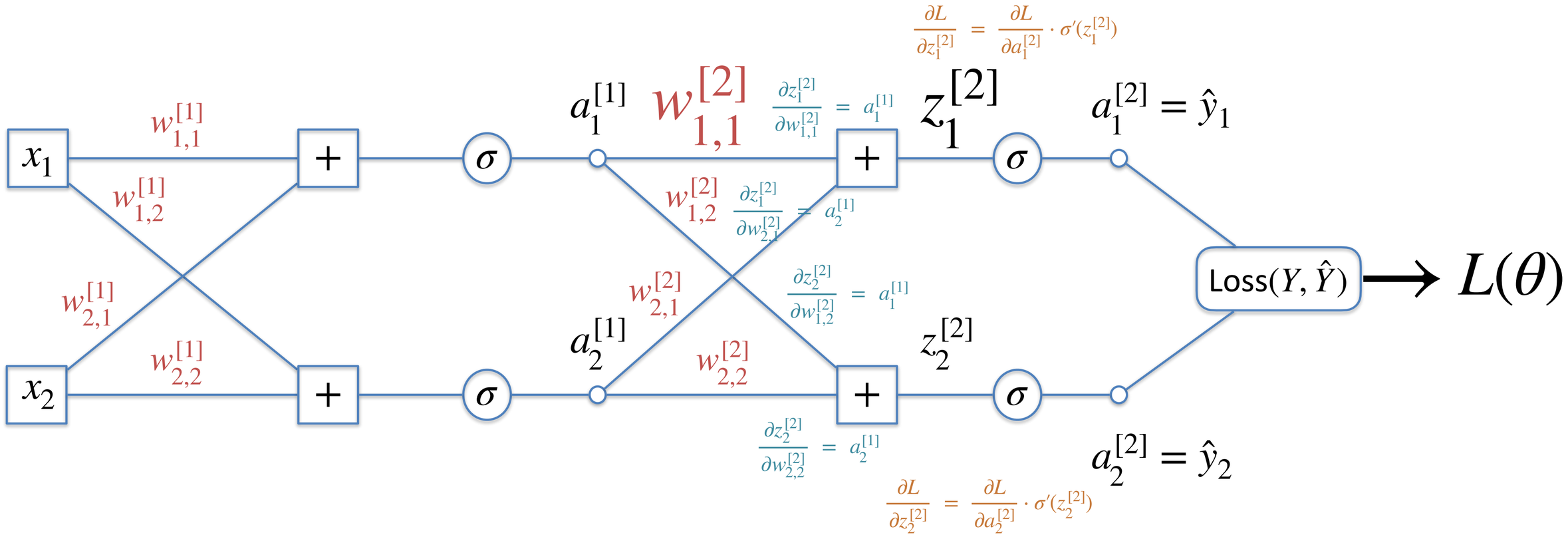
$$\frac{\partial L}{\partial w_{1,1}^{[1]}} = \frac{\partial L}{\partial z_1^{[1]}} \cdot \frac{\partial z_1^{[1]}}{\partial w_{1,1}^{[1]}} \qquad \frac{\partial L}{\partial w_{2,1}^{[1]}} = \frac{\partial L}{\partial z_1^{[1]}} \cdot \frac{\partial z_1^{[1]}}{\partial w_{2,1}^{[1]}} \qquad \frac{\partial L}{\partial w_{1,2}^{[1]}} = \frac{\partial L}{\partial z_2^{[1]}} \cdot \frac{\partial z_2^{[1]}}{\partial w_{1,2}^{[1]}} \qquad \frac{\partial L}{\partial w_{2,2}^{[1]}} = \frac{\partial L}{\partial z_2^{[1]}} \cdot \frac{\partial z_2^{[1]}}{\partial w_{2,2}^{[1]}}$$

Compute Gradient



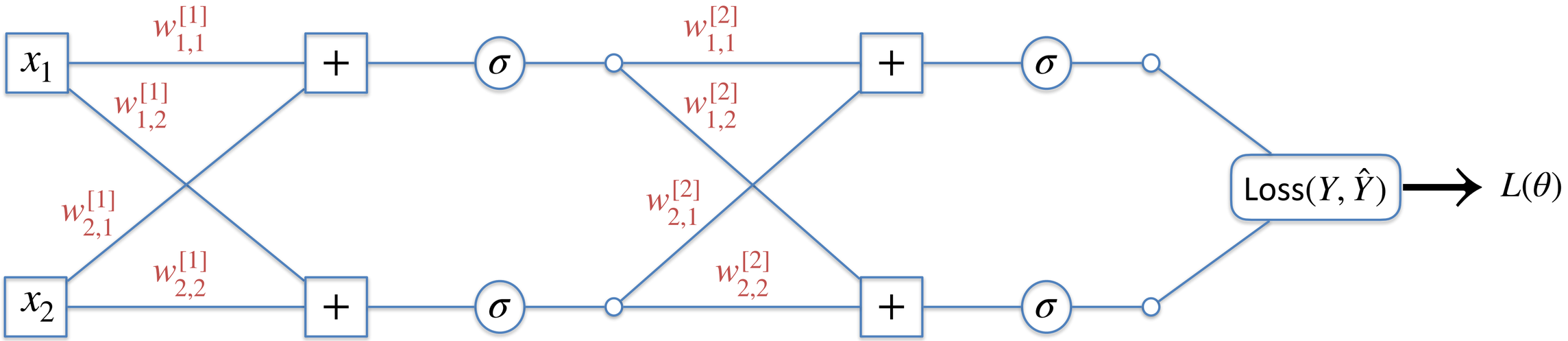
$$\frac{\partial L}{\partial w_{1,1}^{[1]}} = \frac{\partial L}{\partial z_1^{[1]}} \cdot \frac{\partial z_1^{[1]}}{\partial w_{1,1}^{[1]}} \qquad \frac{\partial L}{\partial w_{2,1}^{[1]}} = \frac{\partial L}{\partial z_1^{[1]}} \cdot \frac{\partial z_1^{[1]}}{\partial w_{2,1}^{[1]}} \qquad \frac{\partial L}{\partial w_{1,2}^{[1]}} = \frac{\partial L}{\partial z_2^{[1]}} \cdot \frac{\partial z_2^{[1]}}{\partial w_{1,2}^{[1]}} \qquad \frac{\partial L}{\partial w_{2,2}^{[1]}} = \frac{\partial L}{\partial z_2^{[1]}} \cdot \frac{\partial z_2^{[1]}}{\partial w_{2,2}^{[1]}}$$

$$\frac{\partial L}{\partial w_{1,1}^{[2]}} = \frac{\partial L}{\partial z_1^{[2]}} \cdot \frac{\partial z_1^{[2]}}{\partial w_{1,1}^{[2]}}$$



$$\frac{\partial L}{\partial w_{1,1}^{[1]}} = \frac{\partial L}{\partial z_1^{[1]}} \cdot \frac{\partial z_1^{[1]}}{\partial w_{1,1}^{[1]}} \qquad \frac{\partial L}{\partial w_{2,1}^{[1]}} = \frac{\partial L}{\partial z_1^{[1]}} \cdot \frac{\partial z_1^{[1]}}{\partial w_{2,1}^{[1]}} \qquad \frac{\partial L}{\partial w_{1,2}^{[1]}} = \frac{\partial L}{\partial z_2^{[1]}} \cdot \frac{\partial z_2^{[1]}}{\partial w_{1,2}^{[1]}} \qquad \frac{\partial L}{\partial w_{2,2}^{[1]}} = \frac{\partial L}{\partial z_2^{[1]}} \cdot \frac{\partial z_2^{[1]}}{\partial w_{2,2}^{[1]}}$$

$$\frac{\partial L}{\partial w_{1,1}^{[2]}} = \frac{\partial L}{\partial z_1^{[2]}} \cdot \frac{\partial z_1^{[2]}}{\partial w_{1,1}^{[2]}} \qquad \frac{\partial L}{\partial w_{2,1}^{[2]}} = \frac{\partial L}{\partial z_1^{[2]}} \cdot \frac{\partial z_1^{[2]}}{\partial w_{2,1}^{[2]}} \qquad \frac{\partial L}{\partial w_{1,2}^{[2]}} = \frac{\partial L}{\partial z_2^{[2]}} \cdot \frac{\partial z_2^{[2]}}{\partial w_{1,2}^{[2]}} \qquad \frac{\partial L}{\partial w_{2,2}^{[2]}} = \frac{\partial L}{\partial z_2^{[2]}} \cdot \frac{\partial z_2^{[2]}}{\partial w_{2,2}^{[2]}}$$



$$\frac{\partial L}{\partial w_{1,1}^{[1]}} = \frac{\partial L}{\partial z_1^{[1]}} \cdot \frac{\partial z_1^{[1]}}{\partial w_{1,1}^{[1]}} \qquad \frac{\partial L}{\partial w_{2,1}^{[1]}} = \frac{\partial L}{\partial z_1^{[1]}} \cdot \frac{\partial z_1^{[1]}}{\partial w_{2,1}^{[1]}} \qquad \frac{\partial L}{\partial w_{1,2}^{[1]}} = \frac{\partial L}{\partial z_2^{[1]}} \cdot \frac{\partial z_2^{[1]}}{\partial w_{1,2}^{[1]}} \qquad \frac{\partial L}{\partial w_{2,2}^{[1]}} = \frac{\partial L}{\partial z_2^{[1]}} \cdot \frac{\partial z_2^{[1]}}{\partial w_{2,2}^{[1]}}$$

Optimizers:

- `SGD` (with or without momentum)
- `RMSprop`
- `Adam`
- `Adagrad`
- Etc.

Losses:

- `CategoricalCrossentropy`
- `SparseCategoricalCrossentropy`
- `BinaryCrossentropy`
- `MeanSquaredError`
- `KLDivergence`
- `CosineSimilarity`
- Etc.

Metrics:

- `CategoricalAccuracy`
- `SparseCategoricalAccuracy`
- `BinaryAccuracy`
- `AUC`
- `Precision`
- `Recall`
- Etc.

Quiz questions:

1. What is a tensor?

2. How to compute the gradient of a function?

3. How does the backpropagation algorithm work?